

Universidade Tiradentes
CIÊNCIA DA COMPUTAÇÃO

ANNA CAROLINA SOUZA BISPO
ERICK MARCK BARROS DE MENEZES
LÍVIA PEREIRA GUIMARÃES
MELYSSA MARIA MACEDO TATUM
RAFAEL JOSÉ MECENAS SILVA

**GAME NIGHT PLANNER: UM MODELO CONCEITUAL
PARA A ORGANIZAÇÃO DE SESSÕES DE JOGOS**

**ANNA CAROLINA SOUZA BISPO
ERICK MARCK BARROS DE MENEZES
LÍVIA PEREIRA GUIMARÃES
MELYSSA MARIA MACEDO TATUM
RAFAEL JOSÉ MECENAS SILVA**

**GAME NIGHT PLANNER: UM MODELO CONCEITUAL
PARA A ORGANIZAÇÃO DE SESSÕES DE JOGOS**

ATIVIDADE apresentada como requisito parcial da avaliação da disciplina Programação Avançada, ministrada pelo Prof. Fausto Bernard Melo Soares, no 2º semestre de 2025.

Sumário

1	INTRODUÇÃO	5
2	REFERENCIAL TEÓRICO	6
2.1	A Linguagem de Programação C	6
2.2	Modelagem de Dados com Estruturas (Structs)	7
2.3	Organização de Eventos Sociais e o Papel do Software	7
3	OBJETIVOS	9
3.1	OBJETIVO GERAL	9
3.2	Objetivos Específicos	9
4	METODOLOGIA	10
4.1	Modelagem de Dados e Entidades do Sistema	10
4.1.1	Estrutura Jogador	10
4.1.2	Estrutura Jogo	11
4.1.3	Estrutura SessaoDeJogo	11
4.1.4	Estrutura Partida	12
4.1.5	Estrutura Avaliacao	13
4.1.6	Estrutura Expansao	13
4.1.7	Estrutura Cardapioltem	14
4.1.8	Estrutura Campanha	14
4.1.9	Estrutura Votacao	15
4.1.10	Estrutura Local	15
4.2	Arquitetura do Sistema e Estrutura do Código	16
4.3	Gerenciamento de Dados e Memória	16
4.4	Implementação das Funcionalidades e Lógica de Negócio	16
4.4.1	Validação de Dados e Integridade Referencial	17
4.4.2	Funções Utilitárias	17
4.5	Persistência de Dados	18
4.6	Interface com o Usuário	18
5	RESULTADOS E DISCUSSÕES	19
5.1	Interface Principal e Navegação	19
5.2	Demonstração de Fluxo de Uso: Agendando uma Sessão	19
5.2.1	Passo 1: Cadastro das Entidades Base	20
5.2.2	Passo 2: Agendamento da Sessão e Validação de Integridade	20
5.2.3	Passo 3: Visualização dos Resultados	21

5.3	Discussão dos Resultados	22
5.3.1	Análise das Decisões de Projeto	22
5.3.2	Limitações do Protótipo	23
6	CONSIDERAÇÕES FINAIS	24
	REFERÊNCIAS	26
	APÊNDICE A – APÊNDICE	27

1 INTRODUÇÃO

Os jogos, em seus diversos formatos e estruturas, consolidam-se como instrumentos mediadores cruciais para a interação social. Essa função transcende o mero entretenimento e encontra fundamento na teoria sócio-histórica de Lev Vygotsky, que postula que o desenvolvimento das funções mentais superiores é um processo socialmente construído, o qual se efetiva na relação dialética e na interação entre os sujeitos. A atividade lúdica, ao exigir a aceitação e o cumprimento de regras, atua como um instrumento que não apenas auxilia na produção de conhecimento, mas também gera lazer oriundo da própria atividade e da socialização com o outro. [OLIVEIRA; CHAQUIME, 2022]

Entretanto, a efetivação dessas interações, particularmente para jogos que requerem a presença física ou síncrona dos participantes, depende da superação de desafios logísticos recorrentes. A dificuldade em conciliar horários entre múltiplos indivíduos, a escolha das atividades e a comunicação descentralizada por meio de diversas plataformas representam obstáculos práticos que frequentemente resultam em encontros desorganizados ou cancelados. Essa lacuna organizacional limita a continuidade e o potencial de desenvolvimento social inerente às sessões de jogos. Dessa forma, a modelagem de um sistema dedicado ao agendamento de sessões de jogos surge como uma solução tecnológica estratégica para remover as barreiras logísticas, de forma a garantir a realização contínua dos encontros e a facilitar a interação social e o desenvolvimento dela decorrentemente. [BACKES, 2023]

O diferencial deste trabalho reside na escolha da linguagem de programação C e do paradigma de programação estruturada [PRESSMAN; MAXIM, 2016]. A linguagem C, caracterizada como uma das mais bem-sucedidas e amplamente utilizadas linguagens de alto nível, é notável por sua natureza procedural e por oferecer acesso de baixo nível à memória. Essa seleção visa demonstrar a aplicação de conceitos fundamentais de computação, como a manipulação de memória e a criação de tipos de dados definidos pelo programador — especificamente por meio de estruturas (struct) — na modelagem e resolução de um problema de gerenciamento de informações do mundo real. O emprego da programação estruturada, que restringe o fluxo lógico a três construções básicas (sequência, condição e repetição), contribui para a redução da complexidade dos programas, facilitando sua legibilidade e manutenção.

Portanto, o objetivo deste artigo é apresentar o projeto e o desenvolvimento de um protótipo de sistema para agendamento de sessões de jogos, utilizando a linguagem C. O trabalho detalha a modelagem das entidades principais — jogadores, jogos e sessões — mediante o uso de structs, cujos campos podem ser acessados individualmente, e descreve as funcionalidades implementadas para criar, consultar, atualizar e remover registros, oferecendo, assim, uma solução funcional e didática para o desafio logístico proposto.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta os fundamentos teóricos que sustentam o desenvolvimento do projeto. A exposição está dividida em três áreas principais: a linguagem de programação C, com foco em suas características e relevância para o estudo de estruturas de dados; a modelagem de dados utilizando estruturas (**structs**); e, por fim, o contexto da organização de eventos sociais, que justifica a aplicação prática do sistema.

2.1 A Linguagem de Programação C

Considerada uma das linguagens de alto nível mais influentes, a linguagem C possui um nível de abstração que a aproxima da linguagem humana, distanciando-se do código de máquina. Criada em 1972 por Dennis Ritchie e posteriormente padronizada pelo ANSI, suas características a tornam uma ferramenta robusta tanto para o desenvolvimento de sistemas quanto para o aprendizado de conceitos fundamentais da computação [BACKES, 2023].

A linguagem C é procedural e estruturada, o que permite que um problema complexo seja decomposto em módulos menores e gerenciáveis, onde um programa é essencialmente um conjunto de funções. Sendo uma linguagem compilada, ela é reconhecida por sua eficiência, gerando códigos mais enxutos e rápidos do que muitas outras linguagens. Apesar de ser de alto nível, um de seus maiores diferenciais é a capacidade de acesso de baixo nível à memória, permitindo uma interação mais direta com o hardware. Esta característica de ser "próxima da máquina" exige do programador uma compreensão sobre a organização da memória e o armazenamento de dados [BACKES, 2023; LUCAS, 2025].

A relevância da linguagem C para o aprendizado de estruturas de dados reside em sua natureza minimalista. A linguagem não oferece nativamente estruturas de dados complexas, como listas ou mapas. Portanto, o programador que deseja utilizar tais recursos é obrigado a implementá-los por conta própria, o que força um entendimento profundo do funcionamento "por trás da cortina" desses mecanismos. Essa necessidade de implementação manual é intensificada pelo uso de ponteiros — variáveis que armazenam endereços de memória — e pela responsabilidade do programador no gerenciamento explícito da memória, através de funções como `malloc()` e `free()`. Embora desafiadores, esses conceitos são essenciais para a construção e manutenção de estruturas de dados dinâmicas e eficientes [BACKES, 2023; LUCAS, 2025].

2.2 Modelagem de Dados com Estruturas (Structs)

Em C, as estruturas, ou **structs**, são a ferramenta primária para modelar coleções de dados heterogêneos. Uma **struct** é uma coleção de uma ou mais variáveis agrupadas sob um único nome, que, diferentemente dos arrays, podem ser de tipos distintos. Isso permite a criação de um tipo de dado definido pelo programador, uma abstração que encapsula informações relacionadas de forma lógica e organizada [BACKES, 2023].

O acesso aos membros de uma estrutura é realizado pelo operador ponto (.) para variáveis diretas e pelo operador seta (->) para ponteiros. A linguagem também permite a criação de arrays de estruturas, onde cada elemento do array é uma estrutura completa, uma capacidade fundamental para a implementação do presente projeto. Além disso, as **structs** são a base para a criação de estruturas de dados dinâmicas mais complexas, como as listas ligadas, onde cada nó da lista é uma **struct** que contém tanto o dado quanto o ponteiro para o próximo nó [BACKES, 2023; SIDDIQUI; SIDDIQUI, 2023].

A adequação das **structs** para a modelagem de entidades é evidente. Elas permitem agrupar de maneira coesa os diferentes tipos de dados que descrevem um objeto do mundo real, como um cadastro de pessoa ou, no contexto deste trabalho, as entidades de Jogador, Jogo e Sessão de Jogo, tornando o código mais claro e intuitivo [BACKES, 2023].

2.3 Organização de Eventos Sociais e o Papel do Software

A importância dos jogos e da ludicidade é amplamente reconhecida, especialmente em contextos educacionais e de socialização. A teoria sócio-histórica de Lev Vygotsky, por exemplo, postula que a aprendizagem é um processo essencialmente social, que ocorre na interação entre as pessoas. Nesse sentido, os jogos atuam como importantes instrumentos mediadores, criando um espaço de interação que promove o desenvolvimento [OLIVEIRA; CHAQUIME, 2022; VYGOTSKY, 1999; VYGOTSKI, 2010].

Para que tais interações corram, no entanto, é necessária uma organização logística. É neste ponto que os sistemas de software se tornam relevantes. A engenharia de software tem como um de seus princípios a construção de sistemas para resolver problemas práticos, decompondo problemas complexos em componentes menores e cooperativos. O projeto de um software exige a definição de estruturas de dados e algoritmos capazes de manipular as informações do domínio do problema de forma eficiente [PRESSMAN; MAXIM, 2016; THOMAS; HUNT, 2020].

No contexto deste trabalho, a modelagem das entidades através de **structs** e a implementação de funções para manipulá-las é a tradução direta dos requisitos de um problema social para uma solução técnica. A arquitetura do software, mesmo em um protótipo, busca garantir a coesão funcional dos seus componentes, onde cada função se concentra em uma única tarefa, mas pode ser combinada com outras para criar comportamentos mais

complexos, refletindo as boas práticas da engenharia de software [[PRESSMAN; MAXIM, 2016](#); [BROWN, 2022](#); [DOGLIO, 2022](#)].

3 OBJETIVOS

3.1 OBJETIVO GERAL

O objetivo geral deste trabalho é **desenvolver um protótipo funcional em linguagem C para o gerenciamento e agendamento de sessões de jogos**, baseado em um modelo de dados abrangente e inter-relacionado. O projeto visa demonstrar a aplicação da programação estruturada e da modelagem de dados com **structs** para criar um sistema de linha de comando robusto, com foco na validação de dados e na integridade referencial das informações.

3.2 Objetivos Específicos

Para atingir o objetivo geral proposto, foram definidos e alcançados os seguintes objetivos específicos:

- **Modelar um sistema de dados detalhado** composto por dez estruturas (**structs**) inter-relacionadas, representando as entidades de Jogador, Jogo, SessaoDeJogo, Local, Partida, Avaliacao, Expansao, CardapioItem, Campanha e Votacao.
- **Implementar as funcionalidades de gerenciamento** para cada uma das entidades, permitindo ao usuário cadastrar novos registros e listar as informações existentes de forma detalhada.
- **Empregar uma estratégia de gerenciamento de memória por alocação estática**, utilizando arrays de tamanho fixo e variáveis de controle para armazenar as coleções de dados em tempo de execução.
- **Desenvolver rotinas de validação de dados de entrada e de integridade referencial**, garantindo que as informações inseridas pelo usuário sejam consistentes e que as relações entre as entidades (chaves estrangeiras) sejam válidas.
- **Construir uma interface de usuário (CLI)** totalmente baseada em texto, organizada por meio de menus e submenus hierárquicos para facilitar a navegação e a interação com as funcionalidades do sistema.
- **Documentar a arquitetura do sistema**, o modelo de dados detalhado e as decisões técnicas de implementação no presente trabalho, seguindo as normas da ABNT.

4 METODOLOGIA

O desenvolvimento do protótipo do sistema *Game Night Planner* foi fundamentado no paradigma da programação estruturada, utilizando a linguagem C. A metodologia adotada priorizou uma modelagem de dados abrangente, uma arquitetura modular, uma abordagem de alocação estática de memória e, crucialmente, a implementação de rotinas para validação de dados e garantia da integridade referencial em tempo de execução.

4.1 Modelagem de Dados e Entidades do Sistema

A fundação do sistema reside em um modelo de dados relacional, implementado por meio de dez estruturas (**structs**) distintas. Cada estrutura foi projetada para encapsular um domínio de informação específico e se interconectar com as demais por meio de identificadores numéricos (IDs) para simular um esquema de chaves primárias e estrangeiras. A seguir, cada estrutura é apresentada individualmente, seguida pela descrição de seus atributos.

4.1.1 Estrutura Jogador

Modela o perfil de cada participante e serve como a entidade fundamental de usuário do sistema.

Listing 4.1 – Definição da estrutura Jogador.

```
typedef struct {  
    int idJogador;  
    char nomeCompleto[100];  
    char apelido[50];  
    char email[100];  
    char telefone[20];  
    char generoPreferido[50];  
    char contribuicaoLanche[100];  
    int totalPresencas;  
    float taxaVitorias;  
    char notasObservacao[200];  
} Jogador;
```

idJogador Atua como chave primária para garantir um identificador único para cada jogador.

apelido Armazena um nome de exibição mais curto, utilizado em listagens e menus para facilitar a identificação.

contribuicaoLanche Campo logístico para registrar a contribuição usual do jogador para auxiliar na organização de refeições.

totalPresencas e **taxaVitorias** Atributos que rastreiam o histórico do jogador no grupo para métricas de engajamento e de gamificação.

4.1.2 Estrutura Jogo

Cataloga os jogos disponíveis para o grupo de forma a armazenar metadados e informações logísticas cruciais.

Listing 4.2 – Definição da estrutura Jogo.

```
typedef struct {  
    int idJogo;  
    char titulo[100];  
    int idDono;  
    char genero[50];  
    char plataforma[50];  
    int minJogadores;  
    int maxJogadores;  
    int tempoMedioMinutos;  
    float avaliacaoGrupo;  
    int anoLancamento;  
} Jogo;
```

idJogo Chave primária para identificação única do jogo.

idDono Chave estrangeira que referencia o **idJogador** do proprietário, estabelecendo a relação de posse.

minJogadores, **maxJogadores** Parâmetros essenciais que definem os limites de participantes para uma partida.

tempoMedioMinutos Estimativa de duração, fundamental para o planejamento do cronograma de uma sessão.

4.1.3 Estrutura SessaoDeJogo

A estrutura **SessaoDeJogo** atua como o ponto de convergência onde as entidades abstratas de **Jogador** e **Jogo** são unidas a um contexto logístico específico de tempo e espaço, transformando um planejamento potencial em um encontro concreto e agendado. Sua importância se estende ao servir como âncora para diversas outras funcionalidades, como o registro de **Partida(s)**, a criação de **Votacao(ões)** e a organização do **CardapioItem**, consolidando-a como a entidade mais interconectada do modelo de dados.

Listing 4.3 – Definição da estrutura SessaoDeJogo.

```
typedef struct {
    int idSessao;
    char nomeEvento[100];
    char data[11];
    char horarioInicio[6];
    int idLocal;
    int idAnfitriao;
    int idJogoPrincipal;
    int confirmados[20];
    int numConfirmados;
    char status[50];
} SessaoDeJogo;
```

idSessao Chave primária que identifica unicamente cada sessão agendada.

idLocal, idAnfitriao, idJogoPrincipal Chaves estrangeiras que estabelecem as relações fundamentais do evento: onde, com quem e o quê.

confirmados[] e **numConfirmados** Array de inteiros para armazenar os IDs dos jogadores confirmados e um contador, gerenciando a lista de presença.

status Campo de texto para controlar o ciclo de vida da sessão (e.g., "Planejada", "Confirmada", "Cancelada").

4.1.4 Estrutura Partida

Registra a ocorrência de uma partida específica que aconteceu durante uma sessão de jogo.

Listing 4.4 – Definição da estrutura Partida.

```
typedef struct {
    int idPartida;
    int idSessaoAssociada;
    int idJogo;
    int participantes[20];
    int numParticipantes;
    int idVencedor;
    char pontuacaoFinal[200];
    int duracaoMinutos;
} Partida;
```

idPartida Chave primária para o registro da partida.

idSessaoAssociada Chave estrangeira que vincula a partida à sessão onde ela ocorreu.

idVencedor Armazena o ID do jogador vencedor, permitindo o cálculo de estatísticas. Um valor zero pode indicar empate ou um jogo cooperativo.

4.1.5 Estrutura Avaliacao

Permite que os jogadores registrem suas opiniões e notas sobre os jogos do catálogo.

Listing 4.5 – Definição da estrutura Avaliacao.

```
typedef struct {
    int idAvaliacao;
    int idJogoAvaliado;
    int idJogadorAvaliador;
    float nota;
    char dataAvaliacao[11];
    char comentario[500];
    int recomendaria;
    int ehAnonima;
} Avaliacao;
```

idJogoAvaliado e idJogadorAvaliador Chaves estrangeiras que conectam a avaliação a um jogo específico e ao seu autor.

nota Armazena a nota (e.g., de 0 a 10), que pode ser usada para calcular a **avaliacaoGrupo** na estrutura **Jogo**.

ehAnonima Campo booleano (0 ou 1) que permite ao jogador escolher se sua identidade será revelada na avaliação.

4.1.6 Estrutura Expansao

Modela conteúdos adicionais para um jogo base, tratando-os como entidades separadas, mas vinculadas.

Listing 4.6 – Definição da estrutura Expansao.

```
typedef struct {
    int idExpansao;
    int idJogoBase;
    char tituloExpansao[100];
    int idDono;
    int anoLancamento;
    char oQueAdiciona[200];
    int integraComOutrasExpansoes;
    int requerJogoBase;
} Expansao;
```

idJogoBase Chave estrangeira essencial que vincula a expansão ao seu jogo principal no catálogo.

oQueAdiciona Campo descritivo que resume as novas mecânicas ou componentes introduzidos pela expansão.

4.1.7 Estrutura CardapioItem

Permite a organização detalhada de alimentos e bebidas para uma sessão, facilitando a divisão de tarefas.

Listing 4.7 – Definição da estrutura CardapioItem.

```
typedef struct {  
    int idCardapioItem;  
    int idSessaoAssociada;  
    int idJogadorResponsavel;  
    char nomeItem[100];  
    char quantidade[50];  
    char categoria[50];  
    char notasAdicionais[200];  
    char status[50];  
} CardapioItem;
```

idSessaoAssociada Chave estrangeira que associa o item de cardápio a um evento específico.

idJogadorResponsavel Chave estrangeira que atribui a responsabilidade por levar o item a um participante.

4.1.8 Estrutura Campanha

Agrupa múltiplas sessões de jogo que pertencem a uma mesma narrativa contínua, funcionalidade comum em jogos de RPG.

Listing 4.8 – Definição da estrutura Campanha.

```
typedef struct {  
    int idCampanha;  
    char nomeCampanha[100];  
    int idJogoPrincipal;  
    char descricao[300];  
    int sessoesDaCampanha[15];  
    int numSessoes;  
    char status[50];  
    int idMestreDaCampanha;  
} Campanha;
```

sessoesDaCampanha[] Um array de inteiros que armazena os IDs de todas as sessões que fazem parte da campanha, criando uma relação de um-para-muitos.

idMestreDaCampanha Chave estrangeira que designa um jogador como o narrador ou organizador principal da campanha.

4.1.9 Estrutura Votacao

Modela uma enquete associada a uma sessão, permitindo que os jogadores tomem decisões em grupo de forma democrática.

Listing 4.9 – Definição da estrutura Votacao.

```
typedef struct {
    int idVotacao;
    int idSessaoAssociada;
    char pergunta[200];
    char opcoesTexto[5][100];
    int numOpcoes;
    int votosPorOpcao[5];
    int votantes[20];
    int numVotantes;
    char status[50];
    char dataEncerramento[11];
} Votacao;
```

idSessaoAssociada Vincula a votação a um evento, contextualizando a enquete (e.g., "Qual pizza pedir na sessão de sábado?").

votosPorOpcao[] Array que armazena a contagem de votos para cada opção disponível na enquete.

votantes[] Array para armazenar os IDs dos jogadores que já votaram, evitando votos duplicados.

4.1.10 Estrutura Local

Modela os locais físicos onde as sessões podem ocorrer, centralizando informações de endereço e regras.

Listing 4.10 – Definição da estrutura Local.

```
typedef struct {
    int idLocal;
    char nomeLocal[100];
    char enderecoCompleto[200];
    int idAnfitriaoPadrao;
    char notas[300];
    int capacidadeMaxima;
    char regrasDaCasa[200];
    char telefoneContato[20];
} Local;
```

idLocal Chave primária para o registro do local.

capacidadeMaxima Atributo logístico importante que pode ser usado para validar o número de confirmados em uma sessão.

regrasDaCasa Campo de texto para registrar regras específicas do local (e.g., "Não pisar no tapete branco", "Silêncio após as 22h").

4.2 Arquitetura do Sistema e Estrutura do Código

A arquitetura do sistema foi concebida de forma modular, embora implementada em um único arquivo fonte (`main.c`) para simplificar a compilação e a execução do protótipo. A estrutura lógica é dividida em:

- **Declaração de Tipos e Dados Globais:** O topo do arquivo define todas as `structs` e declara os arrays globais que servirão como base de dados em memória.
- **Protótipos de Funções:** Uma lista de protótipos de todas as funções do sistema é declarada, funcionando como uma interface pública dos módulos.
- **Função Principal (`main`):** Atua como o orquestrador do sistema, responsável por inicializar o programa e manter um laço de repetição que exibe o menu principal e processa a entrada do usuário.
- **Módulos Funcionais:** Cada funcionalidade é encapsulada em funções específicas (e.g., `cadastrarJogador()`, `listarJogadoresDetalhado()`), agrupadas logicamente por funções de gerenciamento (e.g., `gerenciarJogadores()`).

4.3 Gerenciamento de Dados e Memória

Para o armazenamento dos dados em tempo de execução, optou-se por uma abordagem de **alocação estática de memória**. Foram declarados arrays globais de tamanho fixo para cada entidade, com um limite pré-definido pela macro `MAX_ITENS`. Esta escolha, embora limite a escalabilidade, simplifica a implementação ao evitar a complexidade da gestão de memória dinâmica. Uma variável contadora separada (e.g., `numJogadores`) rastreia o número de elementos em uso em cada array.

4.4 Implementação das Funcionalidades e Lógica de Negócio

A lógica de negócio foi implementada através de um conjunto de funções modulares. Um aspecto central da implementação foi a ênfase na robustez e na integridade dos dados, alcançada por meio de validações rigorosas na entrada do usuário.

4.4.1 Validação de Dados e Integridade Referencial

Diferente de uma simples captura de dados, o sistema implementa laços de repetição (**do-while**) nas funções de cadastro para garantir que a entrada do usuário seja não apenas do tipo correto, mas também semanticamente válida. As principais estratégias de validação adotadas foram:

- **Validação de Domínio:** A entrada de dados numéricos é validada para pertencer a um intervalo aceitável. Por exemplo, ao avaliar um jogo, o sistema exige que a nota esteja entre 0.0 e 10.0. Da mesma forma, entradas que simulam valores booleanos são restritas a 0 ou 1.
- **Validação de Consistência:** O sistema verifica a consistência entre diferentes campos de uma mesma entidade. Na função `cadastrarJogoBase()`, por exemplo, é verificado se o `maxJogadores` informado é maior ou igual ao `minJogadores`, prevenindo um estado de dados ilógico.
- **Validação de Integridade Referencial:** Para simular o comportamento de chaves estrangeiras de um banco de dados, o sistema valida a existência de IDs antes de vincular entidades. Ao cadastrar uma sessão, por exemplo, o usuário deve fornecer um `idLocal`, um `idAnfitriao` e um `idJogoPrincipal` que correspondam a registros previamente cadastrados. A validação é feita por meio de funções de busca (e.g., `buscarNomeLocalPorId()`) que verificam se o ID existe. Caso contrário, uma mensagem de erro é exibida e o usuário é solicitado a tentar novamente, garantindo que não sejam criadas referências inválidas.

4.4.2 Funções Utilitárias

Para dar suporte a essas validações e melhorar a qualidade do código, foram desenvolvidas funções auxiliares, tais como:

- **`lerInteiro()` e `lerFloat()`:** Funções encapsuladas para a leitura de dados numéricos que garantem que o usuário digite um valor do tipo correto, tratando entradas inválidas (e.g., texto em vez de número) e limpando o buffer de entrada para evitar erros em leituras subsequentes.
- **Funções de Busca (`buscar...PorId()`):** Uma série de funções que recebem um ID e retornam uma string (e.g., o apelido do jogador, o título do jogo). Elas são o pilar da validação de integridade referencial e também são usadas para exibir informações de forma mais amigável nas listagens.

4.5 Persistência de Dados

Na versão atual do protótipo, não foi implementado um mecanismo de persistência. Toda a informação é mantida em memória RAM e, conseqüentemente, é **volátil**, sendo perdida ao final da execução. A integração de um sistema de salvamento em arquivos é um passo natural para trabalhos futuros.

4.6 Interface com o Usuário

A interação com o sistema ocorre por meio de uma Interface de Linha de Comando (CLI) baseada em menus e submenus hierárquicos. As funções `printf()` e `scanf()` são as principais ferramentas para a exibição de informações e captura de entradas.

5 RESULTADOS E DISCUSSÕES

O resultado final do projeto é um protótipo funcional de linha de comando (CLI), desenvolvido inteiramente na linguagem C. O sistema permite ao usuário interagir com as dez entidades modeladas por meio de uma interface de menus hierárquicos, realizando operações de cadastro e listagem.

5.1 Interface Principal e Navegação

Ao ser executado, o programa exibe uma tela de boas-vindas e o menu principal, que serve como ponto de partida para todas as funcionalidades do sistema, conforme ilustrado na Figura 1. A navegação é baseada na inserção de números correspondentes às opções, e cada opção principal leva a um submenu específico de gerenciamento.

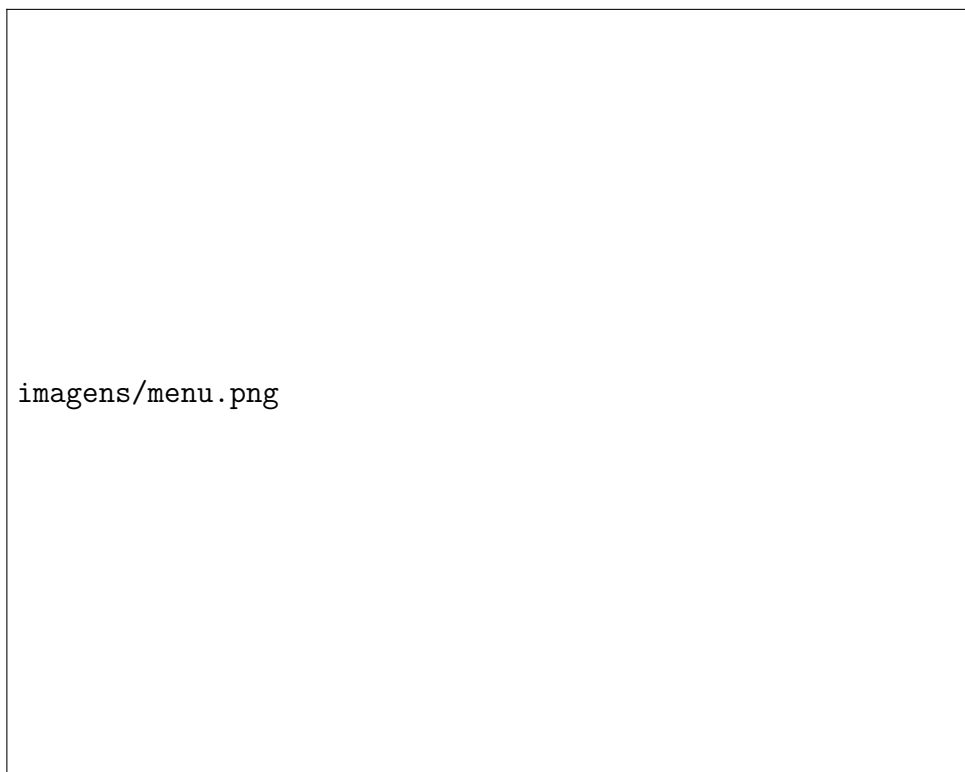


Figura 1 – Tela do menu principal do sistema Game Night Planner.

5.2 Demonstração de Fluxo de Uso: Agendando uma Sessão

Para ilustrar a funcionalidade e a interconexão das entidades, a seguir é demonstrado o fluxo completo para o agendamento de uma nova sessão de jogo, um dos principais casos de uso do sistema.

5.2.1 Passo 1: Cadastro das Entidades Base

Antes de agendar uma sessão, é necessário que as entidades que a compõem já existam no sistema. O primeiro passo consiste em cadastrar um jogador, um jogo e um local. A Figura 2 exibe a tela de cadastro de um novo jogador, onde o sistema solicita o preenchimento dos campos da `struct Jogador`. O mesmo processo é realizado para cadastrar um jogo e um local.



Figura 2 – Tela de interação para o cadastro de um novo jogador.

5.2.2 Passo 2: Agendamento da Sessão e Validação de Integridade

Com as entidades base cadastradas, o usuário pode prosseguir para o agendamento da sessão. O sistema solicita as informações logísticas, como nome do evento e data. Em seguida, para os campos que representam chaves estrangeiras (`idLocal`, `idAnfitriao`, `idJogoPrincipal`), o sistema exibe uma lista simplificada dos registros existentes e exige que o usuário insira um ID válido. A Figura 3 demonstra este processo, evidenciando a rotina de validação de integridade referencial implementada. Se um ID inválido for inserido, o sistema exibe uma mensagem de erro e solicita a entrada novamente.



Figura 3 – Processo de agendamento de uma sessão, com listagem de locais, jogadores e jogos para seleção de IDs válidos.

5.2.3 Passo 3: Visualização dos Resultados

Após o cadastro, as informações podem ser consultadas nas telas de listagem detalhada. A Figura 4 mostra o resultado da listagem de sessões, onde os IDs das entidades relacionadas são exibidos juntamente com seus respectivos nomes ou títulos, recuperados por meio das funções de busca (e.g., `buscarNomeLocalPorId()`).



Figura 4 – Listagem detalhada das sessões agendadas no sistema.

5.3 Discussão dos Resultados

A análise do protótipo desenvolvido permite avaliar o grau de sucesso no atingimento dos objetivos e discutir as decisões técnicas tomadas durante a implementação.

5.3.1 Análise das Decisões de Projeto

A modelagem de dados com dez estruturas inter-relacionadas se mostrou uma abordagem bem-sucedida, permitindo a criação de um sistema com funcionalidades ricas e complexas, como o gerenciamento de campanhas e votações. A simulação de integridade referencial por meio de validações em tempo de execução foi um dos pontos fortes do projeto, garantindo a consistência dos dados mesmo sem o uso de um banco de dados formal.

A escolha pela alocação estática de memória, utilizando arrays de tamanho fixo, foi uma decisão consciente para simplificar o escopo do projeto acadêmico. Embora tenha facilitado a implementação ao evitar a complexidade do gerenciamento de ponteiros com `malloc()` e `realloc()`, esta abordagem representa a principal limitação técnica do protótipo, pois impõe um limite rígido (`MAX_ITENS`) na quantidade de registros que o sistema pode armazenar, tornando-o inviável para uma aplicação real sem modificações.

5.3.2 Limitações do Protótipo

Além da limitação de escalabilidade imposta pela alocação estática, o protótipo apresenta outras limitações que abrem caminho para trabalhos futuros:

- **Volatilidade dos Dados:** O sistema não implementa persistência de dados. Todas as informações são armazenadas em memória RAM e perdidas ao final da execução.
- **Operações CRUD Incompletas:** O foco do desenvolvimento foi nas operações de Criação (Create) e Leitura (Read). As funcionalidades de Atualização (Update) e Exclusão (Delete) não foram implementadas.
- **Interface de Usuário:** A interface de linha de comando, embora funcional, apresenta uma usabilidade limitada em comparação com interfaces gráficas modernas, exigindo que o usuário memorize ou consulte IDs para realizar operações.

Apesar destas limitações, o protótipo cumpre com sucesso seu objetivo principal: demonstrar um modelo de dados funcional e um conjunto de lógicas de negócio robustas para um sistema de agendamento de jogos, servindo como uma excelente base de estudo e para futuras expansões.

6 CONSIDERAÇÕES FINAIS

Este trabalho apresentou o processo de modelagem, especificação e desenvolvimento de um protótipo funcional para um sistema de gerenciamento de sessões de jogos, o *Game Night Planner*. Utilizando a linguagem de programação C e o paradigma estruturado, o projeto buscou traduzir os requisitos de um problema prático de organização social para uma solução de software robusta e bem definida, com foco na consistência e integridade dos dados.

O objetivo geral de desenvolver um protótipo funcional foi plenamente alcançado. O sistema resultante implementa um modelo de dados abrangente, composto por dez estruturas inter-relacionadas, que se mostrou capaz de representar a complexidade das interações envolvidas no agendamento de eventos de jogos. Os objetivos específicos foram igualmente atingidos, resultando em um sistema com funcionalidades de cadastro e listagem para todas as entidades, uma arquitetura modular e uma interface de linha de comando funcional. Um dos resultados mais significativos foi a implementação bem-sucedida de rotinas de validação de dados e de integridade referencial, que garantem a consistência das informações mesmo na ausência de um sistema de gerenciamento de banco de dados formal.

A principal contribuição deste trabalho reside na demonstração prática de como conceitos fundamentais da ciência da computação, como a modelagem de dados com `structs` e o gerenciamento de relações lógicas, podem ser aplicados para criar uma solução completa para um problema do mundo real. O protótipo serve como um estudo de caso detalhado sobre o design de um sistema de informação em um ambiente de baixo nível como o C, destacando tanto as potencialidades quanto as limitações da abordagem escolhida.

Conforme discutido, o protótipo possui limitações inerentes ao seu escopo acadêmico, notadamente a utilização de alocação estática de memória, a ausência de persistência de dados e a incompletude das operações CRUD (sem Atualização e Exclusão). Tais limitações, no entanto, não invalidam os resultados obtidos, mas sim delineiam um caminho claro para futuras melhorias.

Como sugestão para trabalhos futuros, propõe-se a evolução deste protótipo em diversas frentes:

- **Implementação de Persistência de Dados:** Adicionar a capacidade de salvar e carregar os dados em arquivos binários, eliminando a volatilidade das informações.
- **Migração para Alocação Dinâmica de Memória:** Substituir os arrays estáticos por ponteiros e o gerenciamento com `malloc()` e `realloc()`, removendo o limite fixo de registros e tornando o sistema escalável.

- **Desenvolvimento das Funções de Atualização e Exclusão:** Completar o ciclo de operações CRUD, permitindo que o usuário edite e remova registros existentes.
- **Evolução da Interface:** Projetar e implementar uma Interface Gráfica de Usuário (GUI), utilizando bibliotecas como GTK+ ou Qt, para aprimorar a usabilidade e a experiência do usuário.

Em suma, o *Game Night Planner* cumpre seu papel como um protótipo bem-sucedido e uma prova de conceito, oferecendo uma base sólida e bem documentada para o desenvolvimento contínuo de uma ferramenta completa e robusta para a comunidade de jogadores.

REFERÊNCIAS

BACKES, A. *Linguagem C: completa e descomplicada*. 2. ed. Rio de Janeiro: LTC | Livros Técnicos e Científicos Editora Ltda. (GEN Grupo Editorial Nacional S.A.), 2023. 4ª impressão; ISBN 978-85-3528-902-2. ISBN 978-85-3528-902-2.

BROWN, S. *Software Architecture for Developers: Technical leadership and the balance with agility*. [S.l.]: Leanpub, 2022. Versão publicada em 28 de Maio de 2022.

DOGLIO, F. *Skills of a Successful Software Engineer*. Shelter Island, NY: Manning Publications Co., 2022. ISBN 9781617299704.

LUCAS, P. J. *Why Learn C: Go Behind the Curtain of Modern Systems with Paul J. Lucas*. [S.l.]: Apress, 2025. ISBN 9788868815967.

OLIVEIRA, E. de; CHAQUIME, L. P. Vygotsky: Os jogos para além do lúdico. 2022. 13º Congresso de Inovação, Ciência e Tecnologia do IFSP.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. Porto Alegre: AMGH Editora Ltda. (Grupo A Educação), 2016. Tradução da 8ª edição original, com revisão técnica por Reginaldo Arakaki, Julio Arakaki e Renato Manzan de Andrade. ISBN 978-85-8055-534-9.

SIDDIQUI, A.; SIDDIQUI, S. *Data Structure Using C: Theory and Program*. [S.l.: s.n.], 2023. ISBN 9781003453291.

THOMAS, D.; HUNT, A. *The Pragmatic Programmer: your journey to mastery*. Second. [S.l.]: Pearson Education, Inc., 2020. Versão P2.0 (Outubro 28, 2019); Copyright © 2020 Pearson Education, Inc. ISBN 978-0-13-595705-9.

VIGOTSKY, L. S. *A formação social da mente: o desenvolvimento dos processos psicológicos superiores*. 4. ed. São Paulo: Martins Fontes, 1999.

VYGOTSKI, L. S. Aprendizagem e desenvolvimento intelectual na idade escolar. In: VYGOTSKI, L. S.; LURIA, A. R.; LEONTIEV, A. N. (Ed.). *Linguagem, desenvolvimento e aprendizagem*. 11. ed. São Paulo: Ícone, 2010.

APÊNDICE A – APÊNDICE

CÓDIGO FONTE COMPLETO DO SISTEMA

Este apêndice contém o código-fonte completo e integral do protótipo *Game Night Planner*, implementado em um único arquivo ('main.c'), conforme descrito na metodologia deste trabalho.

Listing 1 – Código-fonte completo do protótipo Game Night Planner.

```

1
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6 #define MAX_ITENS 50
7
8 typedef struct {
9     int idJogador;
10    char nomeCompleto[100];
11    char apelido[50];
12    char email[100];
13    char telefone[20];
14    char generoPreferido[50];
15    char contribuicaoLanche[100];
16    int totalPresencas;
17    float taxaVitorias;
18    char notasObservacao[200];
19 } Jogador;
20
21 typedef struct {
22     int idJogo;
23     char titulo[100];
24     int idDono;
25     char genero[50];
26     char plataforma[50];
27     int minJogadores;
28     int maxJogadores;
29     int tempoMedioMinutos;
30     float avaliacaoGrupo;
31     int anoLancamento;
32 } Jogo;
33
34 typedef struct {
35     int idSessao;
36     char nomeEvento[100];
37     char data[15];

```

```
38     char horarioInicio[6];
39     int idLocal;
40     int idAnfitriao;
41     int idJogoPrincipal;
42     int confirmados[20];
43     int numConfirmados;
44     char status[50];
45 } SessaoDeJogo;
46
47 typedef struct {
48     int idPartida;
49     int idSessaoAssociada;
50     int idJogo;
51     int participantes[20];
52     int numParticipantes;
53     int idVencedor;
54     char pontuacaoFinal[200];
55     int duracaoMinutos;
56 } Partida;
57
58 typedef struct {
59     int idAvaliacao;
60     int idJogoAvaliado;
61     int idJogadorAvaliador;
62     float nota;
63     char dataAvaliacao[15];
64     char comentario[500];
65     int recomendaria;
66     int ehAnonima;
67 } Avaliacao;
68
69 typedef struct {
70     int idExpansao;
71     int idJogoBase;
72     char tituloExpansao[100];
73     int idDono;
74     int anoLancamento;
75     char oQueAdiciona[200];
76     int integraComOutrasExpansoes;
77     int requerJogoBase;
78 } Expansao;
79
80 typedef struct {
81     int idCardapioItem;
82     int idSessaoAssociada;
83     int idJogadorResponsavel;
84     char nomeItem[100];
85     char quantidade[50];
```

```
86     char categoria[50];
87     char notasAdicionais[200];
88     char status[50];
89 } CardapioItem;
90
91 typedef struct {
92     int idCampanha;
93     char nomeCampanha[100];
94     int idJogoPrincipal;
95     char descricao[300];
96     int sessoesDaCampanha[15];
97     int numSessoes;
98     char status[50];
99     int idMestreDaCampanha;
100 } Campanha;
101
102 typedef struct {
103     int idVotacao;
104     int idSessaoAssociada;
105     char pergunta[200];
106     char opcoesTexto[5][100];
107     int numOpcoes;
108     int votosPorOpcao[5];
109     int votantes[20];
110     int numVotantes;
111     char status[50];
112     char dataEncerramento[15];
113 } Votacao;
114
115 typedef struct {
116     int idLocal;
117     char nomeLocal[100];
118     char enderecoCompleto[200];
119     int idAnfitriaoPadrao;
120     char notas[300];
121     int capacidadeMaxima;
122     char regrasDaCasa[200];
123     char telefoneContato[20];
124 } Local;
125
126 Jogador listaJogadores[MAX_ITENS];
127 int numJogadores = 0;
128
129 Jogo listaJogos[MAX_ITENS];
130 int numJogos = 0;
131
132 SessaoDeJogo listaSessoes[MAX_ITENS];
133 int numSessoes = 0;
```

```
134
135 Partida listaPartidas[MAX_ITENS];
136 int numPartidas = 0;
137
138 Avaliacao listaAvaliacoes[MAX_ITENS];
139 int numAvaliacoes = 0;
140
141 Expansao listaExpansoes[MAX_ITENS];
142 int numExpansoes = 0;
143
144 CardapioItem listaCardapio[MAX_ITENS * 5];
145 int numCardapioItens = 0;
146
147 Campanha listaCampanhas[MAX_ITENS];
148 int numCampanhas = 0;
149
150 Votacao listaVotacoes[MAX_ITENS];
151 int numVotacoes = 0;
152
153 Local listaLocais[MAX_ITENS];
154 int numLocais = 0;
155
156 void exibirMenuPrincipal();
157 void gerenciarJogadores();
158 void cadastrarJogador();
159 void listarJogadoresDetalhado();
160 void gerenciarCatalogo();
161 void cadastrarJogoBase();
162 void cadastrarExpansao();
163 void listarCatalogoDetalhado();
164 void gerenciarSessoes();
165 void agendarNovaSessao();
166 void listarSessoesDetalhado();
167 void gerenciarCampanhas();
168 void criarNovaCampanha();
169 void listarCampanhasDetalhado();
170 void adicionarSessaoACampanha();
171 void gerenciarLocais();
172 void cadastrarNovoLocal();
173 void listarLocaisDetalhado();
174 void registrarPartida();
175 void avaliarJogo();
176 void gerenciarVotacoes();
177 void criarNovaVotacao();
178 void listarVotacoesAbertas();
179 void votarEmEnquete();
180 void verResultadosVotacao();
181 void gerenciarCardapio();
```

```
182 void adicionarItemAoCardapio();
183 void verCardapioDaSessao();
184 int lerInteiro(const char *prompt);
185 float lerFloat(const char *prompt);
186 void limparBuffer();
187 void pressionarEnterParaContinuar();
188 void removerNovaLinha(char *str);
189 const char *buscarApelidoJogadorPorId(int jogadorId);
190 const char *buscarTituloJogoPorId(int jogoId);
191 const char *buscarNomeLocalPorId(int localId);
192 void listarJogadoresSimples();
193 void listarJogosSimples();
194 void listarExpansoesSimples();
195 void listarSessoesSimples();
196 void listarCampanhasSimples();
197 void listarLocaisSimples();
198
199 int main() {
200     int opcao;
201     printf("Bem-vindo ao Game Night Planner!\n");
202     do {
203         exibirMenuPrincipal();
204         if (scanf("%d", &opcao) != 1) {
205             opcao = -1;
206         }
207         limparBuffer();
208
209         switch (opcao) {
210             case 1:
211                 gerenciarJogadores();
212                 break;
213             case 2:
214                 gerenciarCatalogo();
215                 break;
216             case 3:
217                 gerenciarLocais();
218                 break;
219             case 4:
220                 gerenciarSessoes();
221                 break;
222             case 5:
223                 gerenciarCampanhas();
224                 break;
225             case 6:
226                 registrarPartida();
227                 break;
228             case 7:
229                 avaliarJogo();
```

```

230     break;
231     case 8:
232         gerenciarVotacoes();
233         break;
234     case 9:
235         gerenciarCardapio();
236         break;
237     case 10:
238         printf("\nSaindo do sistema... Ate mais!\n");
239         break;
240     default:
241         printf("\nOpcao invalida! Pressione Enter para tentar novamente.\n
242             ↪ ");
243         pressionarEnterParaContinuar();
244         break;
245     }
246 } while (opcao != 10);
247 return 0;
248 }
249 void exibirMenuPrincipal() {
250     printf("\n+-----+\n");
251     printf("|          GAME NIGHT PLANNER v1.0          |\n");
252     printf("+-----+\n");
253     printf("| [ 1 ] Gerenciar Jogadores          |\n");
254     printf("| [ 2 ] Gerenciar Catalogo de Jogos   |\n");
255     printf("| [ 3 ] Gerenciar Locais             |\n");
256     printf("| [ 4 ] Gerenciar Sessesoes          |\n");
257     printf("| [ 5 ] Gerenciar Campanhas          |\n");
258     printf("| [ 6 ] Registrar Partida Realizada  |\n");
259     printf("| [ 7 ] Avaliar um Jogo              |\n");
260     printf("| [ 8 ] Gerenciar Votacoes          |\n");
261     printf("| [ 9 ] Gerenciar Cardapio de uma Sessao |\n");
262     printf("| [ 10 ] Sair do Programa            |\n");
263     printf("+-----+\n");
264     printf("Escolha uma opcao: ");
265 }
266
267 void gerenciarJogadores() {
268     int subOpcao;
269     do {
270         printf("\n--- Gerenciar Jogadores ---\n");
271         printf("[1] Cadastrar Novo Jogador\n");
272         printf("[2] Listar Jogadores (Detalhado)\n");
273         printf("[3] Voltar ao Menu Principal\n");
274         printf("Escolha uma opcao: ");
275         if (scanf("%d", &subOpcao) != 1) {
276             subOpcao = -1;

```

```
277     }
278     limparBuffer();
279     switch (subOpcao) {
280     case 1:
281         cadastrarJogador();
282         break;
283     case 2:
284         listarJogadoresDetalhado();
285         break;
286     case 3:
287         printf("\nRetornando ao menu principal...\n");
288         break;
289     default:
290         printf("\nOpcao invalida!\n");
291         break;
292     }
293     if (subOpcao != 3)
294         pressionarEnterParaContinuar();
295 } while (subOpcao != 3);
296 }
297
298 void cadastrarJogador() {
299     if (numJogadores >= MAX_ITENS) {
300         printf(
301             "\nNao e possivel cadastrar mais jogadores. Limite maximo
302             ↪ atingido.\n");
303         return;
304     }
305     printf("\n>> Cadastrando Novo Jogador...\n");
306     Jogador novoJogador;
307     novoJogador.idJogador = numJogadores + 1;
308     printf("Nome Completo: ");
309     fgets(novoJogador.nomeCompleto, 100, stdin);
310     removerNovaLinha(novoJogador.nomeCompleto);
311     printf("Apelido: ");
312     fgets(novoJogador.apelido, 50, stdin);
313     removerNovaLinha(novoJogador.apelido);
314     printf("Email: ");
315     fgets(novoJogador.email, 100, stdin);
316     removerNovaLinha(novoJogador.email);
317     printf("Telefone: ");
318     fgets(novoJogador.telefone, 20, stdin);
319     removerNovaLinha(novoJogador.telefone);
320     printf("Genero de Jogo Preferido: ");
321     fgets(novoJogador.generoPreferido, 50, stdin);
322     removerNovaLinha(novoJogador.generoPreferido);
323     printf("Contribuicao de Lanche Comum: ");
324     fgets(novoJogador.contribuicaoLanche, 100, stdin);
```

```
324     removerNovaLinha(novoJogador.contribuicaoLanche);
325     printf("Notas/Observacoes: ");
326     fgets(novoJogador.notasObservacao, 200, stdin);
327     removerNovaLinha(novoJogador.notasObservacao);
328     novoJogador.totalPresencas = 0;
329     novoJogador.taxaVitorias = 0.0f;
330     listaJogadores[numJogadores] = novoJogador;
331     numJogadores++;
332     printf("\nJogador '%s' cadastrado com sucesso com o ID %d!\n",
333           novoJogador.apelido, novoJogador.idJogador);
334 }
335
336 void listarJogadoresDetalhado() {
337     printf("\n--- Lista Detalhada de Jogadores ---\n");
338     if (numJogadores == 0) {
339         printf("Nenhum jogador cadastrado no sistema.\n");
340         return;
341     }
342     for (int i = 0; i < numJogadores; i++) {
343         Jogador jogador = listaJogadores[i];
344         printf("-----\n");
345         printf("ID:                %d\n", jogador.idJogador);
346         printf("Nome Completo:         %s\n", jogador.nomeCompleto);
347         printf("Apelido:                %s\n", jogador.apelido);
348         printf("Email:                 %s\n", jogador.email);
349         printf("Telefone:              %s\n", jogador.telefone);
350         printf("Genero Preferido:       %s\n", jogador.generoPreferido);
351         printf("Contribuicao Comum:      %s\n", jogador.contribuicaoLanche);
352         printf("Observacoes:           %s\n", jogador.notasObservacao);
353         printf("Total de Presencas:     %d\n", jogador.totalPresencas);
354         printf("Taxa de Vitorias:       %.2f%%\n", jogador.taxaVitorias);
355     }
356     printf("-----\n");
357 }
358
359 void gerenciarCatalogo() {
360     int subOpcao;
361     do {
362         printf("\n--- Gerenciar Catalogo de Jogos ---\n");
363         printf("[1] Cadastrar Novo Jogo Base\n");
364         printf("[2] Adicionar Expansao a um Jogo\n");
365         printf("[3] Listar Catalogo (Detalhado)\n");
366         printf("[4] Voltar ao Menu Principal\n");
367         printf("Escolha uma opcao: ");
368         if (scanf("%d", &subOpcao) != 1) {
369             subOpcao = -1;
370         }
371         limparBuffer();
```

```
372     switch (subOpcao) {
373     case 1:
374         cadastrarJogoBase();
375         break;
376     case 2:
377         cadastrarExpansao();
378         break;
379     case 3:
380         listarCatalogoDetalhado();
381         break;
382     case 4:
383         printf("\nRetornando ao menu principal...\n");
384         break;
385     default:
386         printf("\nOpcao invalida!\n");
387         break;
388     }
389     if (subOpcao != 4)
390         pressionarEnterParaContinuar();
391 } while (subOpcao != 4);
392 }
393
394 void cadastrarJogoBase() {
395     if (numJogos >= MAX_ITENS) {
396         printf("\nNao e possivel cadastrar mais jogos. Limite maximo
397             ↪ atingido.\n");
398         return;
399     }
400     if (numJogadores == 0) {
401         printf("\nErro: E necessario cadastrar ao menos um jogador para ser
402             ↪ o dono do jogo.\n");
403         return;
404     }
405     printf("\n>> Cadastrando Novo Jogo Base...\n");
406     Jogo novoJogo;
407     novoJogo.idJogo = numJogos + 1;
408
409     printf("Titulo do Jogo: ");
410     fgets(novoJogo.titulo, 100, stdin);
411     removerNovaLinha(novoJogo.titulo);
412
413     printf("Genero: ");
414     fgets(novoJogo.genero, 50, stdin);
415     removerNovaLinha(novoJogo.genero);
416
417     printf("Plataforma (PC, PS5, Tabuleiro, etc.): ");
418     fgets(novoJogo.plataforma, 50, stdin);
```

```
418 removerNovaLinha(novoJogo.plataforma);
419
420 do {
421     novoJogo.minJogadores = lerInteiro("Numero minimo de jogadores: ");
422     if (novoJogo.minJogadores <= 0) {
423         printf("Erro: 0 numero minimo deve ser pelo menos 1.\n");
424     }
425 } while (novoJogo.minJogadores <= 0);
426
427 do {
428     novoJogo.maxJogadores = lerInteiro("Numero maximo de jogadores: ");
429     if (novoJogo.maxJogadores < novoJogo.minJogadores) {
430         printf("Erro: 0 numero maximo (%d) nao pode ser menor que o
431             ↪ minimo (%d).\n",
432                 novoJogo.maxJogadores, novoJogo.minJogadores);
433     }
434 } while (novoJogo.maxJogadores < novoJogo.minJogadores);
435
436 do {
437     novoJogo.tempoMedioMinutos = lerInteiro("Tempo medio de partida (
438         ↪ minutos): ");
439     if (novoJogo.tempoMedioMinutos <= 0) {
440         printf("Erro: 0 tempo medio deve ser maior que 0.\n");
441     }
442 } while (novoJogo.tempoMedioMinutos <= 0);
443
444 novoJogo.anoLancamento = lerInteiro("Ano de lancamento: ");
445
446 listarJogadoresSimples();
447
448 int donoValido = 0;
449 do {
450     novoJogo.idDono = lerInteiro("Digite o ID do Dono do Jogo: ");
451
452     if (strcmp(buscarApelidoJogadorPorId(novoJogo.idDono), "Nao
453         ↪ encontrado") != 0) {
454         donoValido = 1;
455     } else {
456         printf("Erro: Jogador com ID %d nao encontrado. Tente novamente
457             ↪ .\n", novoJogo.idDono);
458     }
459 } while (!donoValido);
460
461 novoJogo.avaliacaoGrupo = 0.0f;
462 listaJogos[numJogos] = novoJogo;
463 numJogos++;
464 printf("\nJogo '%s' cadastrado com sucesso com o ID %d!\n", novoJogo.
465     ↪ titulo,
```

```
461         novoJogo.idJogo);
462     }
463
464     void cadastrarExpansao() {
465         if (numExpansoes >= MAX_ITENS) {
466             printf("\nNao e possivel cadastrar mais expansoes. Limite maximo
467                 ↳ atingido.\n");
468             return;
469         }
470         if (numJogos == 0) {
471             printf("\nErro: E necessario cadastrar um jogo base antes de
472                 ↳ adicionar uma expansao.\n");
473             return;
474         }
475
476         printf("\n>> Cadastrando Novo Expansao...\n");
477         Expansao novaExpansao;
478         novaExpansao.idExpansao = numExpansoes + 1;
479
480         int jogoValido = 0;
481         do {
482             listarJogosSimples();
483             novaExpansao.idJogoBase = lerInteiro("Digite o ID do Jogo Base para
484                 ↳ esta expansao: ");
485
486             if (strcmp(buscarTituloJogoPorId(novaExpansao.idJogoBase), "Jogo
487                 ↳ base nao encontrado") != 0) {
488                 jogoValido = 1;
489             } else {
490                 printf("Erro: Jogo com ID %d nao encontrado. Tente novamente.\n"
491                     ↳ , novaExpansao.idJogoBase);
492             }
493         } while (!jogoValido);
494
495         printf("Titulo da Expansao: ");
496         fgets(novaExpansao.tituloExpansao, 100, stdin);
497         removerNovaLinha(novaExpansao.tituloExpansao);
498
499         printf("O que a expansao adiciona (resumo): ");
500         fgets(novaExpansao.oQueAdiciona, 200, stdin);
501         removerNovaLinha(novaExpansao.oQueAdiciona);
502
503         int donoValido = 0;
504         do {
505             listarJogadoresSimples();
506             novaExpansao.idDono = lerInteiro("Digite o ID do Dono da Expansao: "
507                 ↳ );
508             if (strcmp(buscarApelidoJogadorPorId(novaExpansao.idDono), "Nao
```

```
    ↪ encontrado") != 0) {
503     donoValido = 1;
504 } else {
505     printf("Erro: Jogador com ID %d nao encontrado.\n", novaExpansao
        ↪ .idDono);
506 }
507 } while (!donoValido);
508
509 novaExpansao.anoLancamento = lerInteiro("Ano de lancamento da expansao
    ↪ : ");
510
511 do {
512     novaExpansao.requerJogoBase = lerInteiro("Requer o jogo base para
        ↪ jogar? (1=Sim, 0=Nao): ");
513     if (novaExpansao.requerJogoBase < 0 || novaExpansao.requerJogoBase >
        ↪ 1) {
514         printf("Erro: Opcao invalida. Digite 1 para Sim ou 0 para Nao.\n
            ↪ ");
515     }
516 } while (novaExpansao.requerJogoBase < 0 || novaExpansao.
    ↪ requerJogoBase > 1);
517
518 do {
519     novaExpansao.integraComOutrasExpansoes = lerInteiro("Integra com
        ↪ outras expansoes? (1=Sim, 0=Nao): ");
520     if (novaExpansao.integraComOutrasExpansoes < 0 || novaExpansao.
        ↪ integraComOutrasExpansoes > 1) {
521         printf("Erro: Opcao invalida. Digite 1 para Sim ou 0 para Nao.\n
            ↪ ");
522     }
523 } while (novaExpansao.integraComOutrasExpansoes < 0 || novaExpansao.
    ↪ integraComOutrasExpansoes > 1);
524
525 listaExpansoes[numExpansoes] = novaExpansao;
526 numExpansoes++;
527 printf("\nExpansao '%s' cadastrada com sucesso!\n", novaExpansao.
    ↪ tituloExpansao);
528 }
529
530 void listarCatalogoDetalhado() {
531     printf("\n--- Catalogo Detalhado de Jogos e Expansoes ---\n");
532     printf("\n--- JOGOS BASE ---\n");
533     if (numJogos == 0) {
534         printf("Nenhum jogo base cadastrado.\n");
535     } else {
536         for (int i = 0; i < numJogos; i++) {
537             Jogo jogo = listaJogos[i];
538             printf("-----\n");
```

```

539     printf("ID do Jogo:           %d\n", jogo.idJogo);
540     printf("Titulo:              %s\n", jogo.titulo);
541     printf("Dono:                ID %d (%s)\n", jogo.idDono,
542           buscarApelidoJogadorPorId(jogo.idDono));
543     printf("Genero:              %s\n", jogo.genero);
544     printf("Plataforma:          %s\n", jogo.plataforma);
545     printf("Jogadores:           %d a %d\n", jogo.minJogadores,
546           jogo.maxJogadores);
547     printf("Tempo Medio:         %d min\n", jogo.tempoMedioMinutos);
548     printf("Ano:                 %d\n", jogo.anoLancamento);
549     printf("Avaliacao do Grupo:  %.1f / 10.0\n", jogo.avaliacaoGrupo);
550 }
551 }
552 printf("\n--- EXPANSOES ---\n");
553 if (numExpansoes == 0) {
554     printf("Nenhuma expansao cadastrada.\n");
555 } else {
556     for (int i = 0; i < numExpansoes; i++) {
557         Expansao exp = listaExpansoes[i];
558         printf("-----\n");
559         printf("ID da Expansao:           %d\n", exp.idExpansao);
560         printf("Titulo:              %s\n", exp.tituloExpansao);
561         printf("Expansao para:           ID %d (%s)\n", exp.idJogoBase,
562               buscarTituloJogoPorId(exp.idJogoBase));
563         printf("Dono:                ID %d (%s)\n", exp.idDono,
564               buscarApelidoJogadorPorId(exp.idDono));
565         printf("Adiciona:              %s\n", exp.oQueAdiciona);
566     }
567 }
568 printf("-----\n");
569 }
570
571 void gerenciarSessoes() {
572     int subOpcao;
573     do {
574         printf("\n--- Gerenciar Sessoes de Jogo ---\n");
575         printf("[1] Agendar Nova Sessao\n");
576         printf("[2] Listar Sessoes Agendadas (Detalhado)\n");
577         printf("[3] Voltar ao Menu Principal\n");
578         printf("Escolha uma opcao: ");
579         if (scanf("%d", &subOpcao) != 1) {
580             subOpcao = -1;
581         }
582         limparBuffer();
583         switch (subOpcao) {
584             case 1:
585                 agendarNovaSessao();
586                 break;

```

```
587     case 2:
588         listarSessoesDetalhado();
589         break;
590     case 3:
591         printf("\nRetornando ao menu principal...\n");
592         break;
593     default:
594         printf("\nOpcao invalida!\n");
595         break;
596     }
597     if (subOpcao != 3)
598         pressionarEnterParaContinuar();
599 } while (subOpcao != 3);
600 }
601
602 void agendarNovaSessao() {
603     if (numSessoes >= MAX_ITENS) {
604         printf("\nNao e possivel agendar mais sessoes. Limite maximo
605             ↪ atingido.\n");
606         return;
607     }
608     if (numJogadores == 0 || numJogos == 0 || numLocais == 0) {
609         printf("\nErro: E necessario ter ao menos um Jogador, Jogo e Local
610             ↪ cadastrado para agendar uma sessao.\n");
611         return;
612     }
613
614     printf("\n>> Agendando Nova Sessao de Jogo...\n");
615     SessaoDeJogo novaSessao;
616     novaSessao.idSessao = numSessoes + 1;
617
618     printf("Nome do Evento: ");
619     fgets(novaSessao.nomeEvento, 100, stdin);
620     removerNovaLinha(novaSessao.nomeEvento);
621
622     printf("Data (DD/MM/AAAA): ");
623     fgets(novaSessao.data, 15, stdin);
624     removerNovaLinha(novaSessao.data);
625
626     printf("Horario de Inicio (HH:MM): ");
627     fgets(novaSessao.horarioInicio, 6, stdin);
628     removerNovaLinha(novaSessao.horarioInicio);
629
630     int localValido = 0;
631     do {
632         listarLocaisSimples();
633         novaSessao.idLocal = lerInteiro("Digite o ID do Local do evento: ");
```

```
633     if (strcmp(buscarNomeLocalPorId(novaSessao.idLocal), "Local nao
        ↳ encontrado") != 0) {
634         localValido = 1;
635     } else {
636         printf("Erro: Local com ID %d nao encontrado. Tente novamente.\n
        ↳ ", novaSessao.idLocal);
637     }
638 } while (!localValido);
639
640 int anfitriaoValido = 0;
641 do {
642     listarJogadoresSimples();
643     novaSessao.idAnfitriao = lerInteiro("Digite o ID do Anfitriao: ");
644
645     if (strcmp(buscarApelidoJogadorPorId(novaSessao.idAnfitriao), "Nao
        ↳ encontrado") != 0) {
646         anfitriaoValido = 1;
647     } else {
648         printf("Erro: Jogador com ID %d nao encontrado. Tente novamente
        ↳ .\n", novaSessao.idAnfitriao);
649     }
650 } while (!anfitriaoValido);
651
652 int jogoValido = 0;
653 do {
654     listarJogosSimples();
655     novaSessao.idJogoPrincipal = lerInteiro("Digite o ID do Jogo
        ↳ Principal: ");
656
657     if (strcmp(buscarTituloJogoPorId(novaSessao.idJogoPrincipal), "Jogo
        ↳ base nao encontrado") != 0) {
658         jogoValido = 1;
659     } else {
660         printf("Erro: Jogo com ID %d nao encontrado. Tente novamente.\n"
        ↳ , novaSessao.idJogoPrincipal);
661     }
662 } while (!jogoValido);
663
664 strcpy(novaSessao.status, "Planejada");
665 novaSessao.numConfirmados = 0;
666 listaSesoes[numSesoes] = novaSessao;
667 numSesoes++;
668 printf("\nSessao '%s' agendada com sucesso com o ID %d!\n",
        novaSessao.nomeEvento, novaSessao.idSessao);
669 }
670
671
672 void listarSesoesDetalhado() {
673     printf("\n--- Lista Detalhada de Sesoes Agendadas ---\n");
```

```
674     if (numSesoes == 0) {
675         printf("Nenhuma sessao agendada no momento.\n");
676         return;
677     }
678     for (int i = 0; i < numSesoes; i++) {
679         SessaoDeJogo sessao = listaSesoes[i];
680         printf("-----\n");
681         printf("ID da Sessao:          %d\n", sessao.idSessao);
682         printf("Evento:                %s\n", sessao.nomeEvento);
683         printf("Data e Hora:            %s as %s\n", sessao.data,
684             sessao.horarioInicio);
685         printf("Status:                %s\n", sessao.status);
686         printf("Local:                ID %d (%s)\n", sessao.idLocal,
687             buscarNomeLocalPorId(sessao.idLocal));
688         printf("Anfitriao:            ID %d (%s)\n", sessao.idAnfitriao,
689             buscarApelidoJogadorPorId(sessao.idAnfitriao));
690         printf("Jogo Principal:        ID %d (%s)\n", sessao.idJogoPrincipal,
691             buscarTituloJogoPorId(sessao.idJogoPrincipal));
692         printf("Jogadores Confirmados: %d\n", sessao.numConfirmados);
693     }
694     printf("-----\n");
695 }
696
697 void gerenciarCampanhas() {
698     int subOpcao;
699     do {
700         printf("\n--- Gerenciar Campanhas ---\n");
701         printf("[1] Criar Nova Campanha\n");
702         printf("[2] Listar Campanhas (Detalhado)\n");
703         printf("[3] Adicionar Sessao a uma Campanha\n");
704         printf("[4] Voltar ao Menu Principal\n");
705         printf("Escolha uma opcao: ");
706         if (scanf("%d", &subOpcao) != 1) {
707             subOpcao = -1;
708         }
709         limparBuffer();
710         switch (subOpcao) {
711             case 1:
712                 criarNovaCampanha();
713                 break;
714             case 2:
715                 listarCampanhasDetalhado();
716                 break;
717             case 3:
718                 adicionarSessaoACampanha();
719                 break;
720             case 4:
721                 printf("\nRetornando ao menu principal...\n");
```

```
722     break;
723     default:
724         printf("\nOpcao invalida!\n");
725         break;
726     }
727     if (subOpcao != 4)
728         pressionarEnterParaContinuar();
729 } while (subOpcao != 4);
730 }
731
732 void criarNovaCampanha() {
733     if (numCampanhas >= MAX_ITENS) {
734         printf("\nNao e possivel criar mais campanhas. Limite maximo
735             ↪ atingido.\n");
736         return;
737     }
738     if (numJogadores == 0 || numJogos == 0) {
739         printf("\nErro: E necessario ter ao menos um Jogador (para ser o
740             ↪ mestre) e um Jogo cadastrado.\n");
741         return;
742     }
743     printf("\n>> Criando Nova Campanha...\n");
744     Campanha novaCampanha;
745     novaCampanha.idCampanha = numCampanhas + 1;
746
747     printf("Nome da Campanha: ");
748     fgets(novaCampanha.nomeCampanha, 100, stdin);
749     removerNovaLinha(novaCampanha.nomeCampanha);
750
751     printf("Breve descricao da Campanha: ");
752     fgets(novaCampanha.descricao, 300, stdin);
753     removerNovaLinha(novaCampanha.descricao);
754
755     int jogoValido = 0;
756     do {
757         listarJogosSimples();
758         novaCampanha.idJogoPrincipal = lerInteiro("Digite o ID do Jogo
759             ↪ Principal da Campanha: ");
760
761         if (strcmp(buscarTituloJogoPorId(novaCampanha.idJogoPrincipal), "
762             ↪ Jogo base nao encontrado") != 0) {
763             jogoValido = 1;
764         } else {
765             printf("Erro: Jogo com ID %d nao encontrado. Tente novamente.\n"
766                 ↪ , novaCampanha.idJogoPrincipal);
767         }
768     } while (!jogoValido);
```

```

765
766     int mestreValido = 0;
767     do {
768         listarJogadoresSimples();
769         novaCampanha.idMestreDaCampanha = lerInteiro("Digite o ID do Jogador
           ↳ que sera o Mestre/Organizador: ");
770
771         if (strcmp(buscarApelidoJogadorPorId(novaCampanha.idMestreDaCampanha
           ↳ ), "Nao encontrado") != 0) {
772             mestreValido = 1;
773         } else {
774             printf("Erro: Jogador com ID %d nao encontrado. Tente novamente
           ↳ .\n", novaCampanha.idMestreDaCampanha);
775         }
776     } while (!mestreValido);
777
778     strcpy(novaCampanha.status, "Em Andamento");
779     novaCampanha.numSesoes = 0;
780     listaCampanhas[numCampanhas] = novaCampanha;
781     numCampanhas++;
782     printf("\nCampanha '%s' criada com sucesso com o ID %d!\n",
783           novaCampanha.nomeCampanha, novaCampanha.idCampanha);
784 }
785
786 void listarCampanhasDetalhado() {
787     printf("\n--- Lista Detalhada de Campanhas ---\n");
788     if (numCampanhas == 0) {
789         printf("Nenhuma campanha criada no momento.\n");
790         return;
791     }
792     for (int i = 0; i < numCampanhas; i++) {
793         Campanha campanha = listaCampanhas[i];
794         printf("-----\n");
795         printf("ID da Campanha:      %d\n", campanha.idCampanha);
796         printf("Nome:                %s\n", campanha.nomeCampanha);
797         printf("Descricao:           %s\n", campanha.descricao);
798         printf("Status:              %s\n", campanha.status);
799         printf("Jogo Principal:      ID %d (%s)\n", campanha.idJogoPrincipal
           ↳ ,
800               buscarTituloJogoPorId(campanha.idJogoPrincipal));
801         printf("Mestre/Organizador:  ID %d (%s)\n", campanha.
           ↳ idMestreDaCampanha,
802               buscarApelidoJogadorPorId(campanha.idMestreDaCampanha));
803         printf("Sesoes Registradas: %d\n", campanha.numSesoes);
804         if (campanha.numSesoes > 0) {
805             printf("IDs das Sesoes:      ");
806             for (int j = 0; j < campanha.numSesoes; j++) {
807                 printf("%d ", campanha.sesoesDaCampanha[j]);

```

```
808     }
809     printf("\n");
810 }
811 }
812 printf("-----\n");
813 }
814
815 void adicionarSessaoACampanha() {
816     if (numCampanhas == 0 || numSesoes == 0) {
817         printf("\nErro: E necessario ter ao menos uma campanha e uma sessao
818             ↪ para fazer a vinculacao.\n");
819         return;
820     }
821
822     printf("\n>> Vinculando Sessao a uma Campanha...\n");
823     int idCampanha, idSessao;
824     int indiceCampanha = -1;
825
826     do {
827         listarCampanhasSimples();
828         idCampanha = lerInteiro("Digite o ID da Campanha: ");
829
830         indiceCampanha = -1;
831         for (int i = 0; i < numCampanhas; i++) {
832             if (listaCampanhas[i].idCampanha == idCampanha) {
833                 indiceCampanha = i;
834                 break;
835             }
836         }
837
838         if (indiceCampanha == -1) {
839             printf("Erro: Campanha com ID %d nao encontrada. Tente novamente
840                 ↪ .\n", idCampanha);
841         }
842     } while (indiceCampanha == -1);
843
844     int sessaoEncontrada = 0;
845     do {
846         listarSesoesSimples();
847         idSessao = lerInteiro("Digite o ID da Sessao a ser adicionada: ");
848
849         sessaoEncontrada = 0;
850         for (int i = 0; i < numSesoes; i++) {
851             if (listaSesoes[i].idSessao == idSessao) {
852                 sessaoEncontrada = 1;
853                 break;
854             }
855         }
856     }
```

```
854
855     if (!sessaoEncontrada) {
856         printf("Erro: Sessao com ID %d nao encontrada. Tente novamente.\n
            ↪ n", idSessao);
857     }
858 } while (!sessaoEncontrada);
859
860 if (listaCampanhas[indiceCampanha].numSesseoes < 15) {
861     listaCampanhas[indiceCampanha]
862         .sesseoesDaCampanha[listaCampanhas[indiceCampanha].numSesseoes] =
            ↪ idSessao;
863     listaCampanhas[indiceCampanha].numSesseoes++;
864     printf("\nSessao %d adicionada a campanha '%s' com sucesso!\n",
            ↪ idSessao,
865         listaCampanhas[indiceCampanha].nomeCampanha);
866 } else {
867     printf("\nErro: Esta campanha ja atingiu o numero maximo de sesseoes
            ↪ (15).\n");
868 }
869 }
870
871 void gerenciarLocais() {
872     int subOpcao;
873     do {
874         printf("\n--- Gerenciar Locais de Encontro ---\n");
875         printf("[1] Cadastrar Novo Local\n");
876         printf("[2] Listar Locais Cadastrados (Detalhado)\n");
877         printf("[3] Voltar ao Menu Principal\n");
878         printf("Escolha uma opcao: ");
879         if (scanf("%d", &subOpcao) != 1) {
880             subOpcao = -1;
881         }
882         limparBuffer();
883         switch (subOpcao) {
884             case 1:
885                 cadastrarNovoLocal();
886                 break;
887             case 2:
888                 listarLocaisDetalhado();
889                 break;
890             case 3:
891                 printf("\nRetornando ao menu principal...\n");
892                 break;
893             default:
894                 printf("\nOpcao invalida!\n");
895                 break;
896         }
897         if (subOpcao != 3)
```

```
898     pressionarEnterParaContinuar();
899 } while (subOpcao != 3);
900 }
901
902 void cadastrarNovoLocal() {
903     if (numLocais >= MAX_ITENS) {
904         printf("\nNao e possivel cadastrar mais locais. Limite maximo
          ↪ atingido.\n");
905         return;
906     }
907     printf("\n>> Cadastrando Novo Local...\n");
908     Local novoLocal;
909     novoLocal.idLocal = numLocais + 1;
910
911     printf("Nome do Local: ");
912     fgets(novoLocal.nomeLocal, 100, stdin);
913     removerNovaLinha(novoLocal.nomeLocal);
914
915     printf("Endereco Completo: ");
916     fgets(novoLocal.enderecoCompleto, 200, stdin);
917     removerNovaLinha(novoLocal.enderecoCompleto);
918
919     do {
920         novoLocal.capacidadeMaxima = lerInteiro("Capacidade Maxima de
          ↪ Pessoas: ");
921         if (novoLocal.capacidadeMaxima <= 0) {
922             printf("Erro: A capacidade maxima deve ser de pelo menos 1
          ↪ pessoa.\n");
923         }
924     } while (novoLocal.capacidadeMaxima <= 0);
925
926     printf("Telefone de Contato (opcional): ");
927     fgets(novoLocal.telefoneContato, 20, stdin);
928     removerNovaLinha(novoLocal.telefoneContato);
929
930     printf("Regras da Casa: ");
931     fgets(novoLocal.regrasDaCasa, 200, stdin);
932     removerNovaLinha(novoLocal.regrasDaCasa);
933
934     printf("Notas Adicionais: ");
935     fgets(novoLocal.notas, 300, stdin);
936     removerNovaLinha(novoLocal.notas);
937
938     int anfitriaoValido = 0;
939     do {
940         listarJogadoresSimples();
941         novoLocal.idAnfitriaoPadrao = lerInteiro("Digite o ID do Anfitriao
          ↪ Padrao (ou 0 se nao houver): ");
```

```

942
943     if (novoLocal.idAnfitriaoPadrao == 0) {
944         anfitriaoValido = 1;
945     } else {
946         if (strcmp(buscarApelidoJogadorPorId(novoLocal.idAnfitriaoPadrao
947             ↪ ), "Nao encontrado") != 0) {
948             anfitriaoValido = 1;
949         } else {
950             printf("Erro: Jogador com ID %d nao encontrado. Digite um ID
951             ↪ valido ou 0.\n", novoLocal.idAnfitriaoPadrao);
952         }
953     }
954     while (!anfitriaoValido);
955
956     listaLocais[numLocais] = novoLocal;
957     numLocais++;
958     printf("\nLocal '%s' cadastrado com sucesso com o ID %d!\n",
959         novoLocal.nomeLocal, novoLocal.idLocal);
960 }
961
962 void listarLocaisDetalhado() {
963     printf("\n--- Lista Detalhada de Locais Cadastrados ---\n");
964     if (numLocais == 0) {
965         printf("Nenhum local cadastrado no sistema.\n");
966         return;
967     }
968     for (int i = 0; i < numLocais; i++) {
969         Local local = listaLocais[i];
970         printf("-----\n");
971         printf("ID do Local:          %d\n", local.idLocal);
972         printf("Nome:                %s\n", local.nomeLocal);
973         printf("Endereco:             %s\n", local.enderecoCompleto);
974         printf("Capacidade Maxima:    %d pessoas\n", local.capacidadeMaxima);
975         if (local.idAnfitriaoPadrao > 0) {
976             printf("Anfitriao Padrao:     ID %d (%s)\n", local.
977             ↪ idAnfitriaoPadrao,
978                 buscarApelidoJogadorPorId(local.idAnfitriaoPadrao));
979         } else {
980             printf("Anfitriao Padrao:     Nenhum\n");
981         }
982         printf("Telefone de Contato: %s\n", local.telefoneContato);
983         printf("Regras da Casa:      %s\n", local.regrasDaCasa);
984         printf("Notas:               %s\n", local.notas);
985     }
986     printf("-----\n");
987 }
988
989 void registrarPartida() {

```

```
987     if (numPartidas >= MAX_ITENS) {
988         printf("\nNao e possivel registrar mais partidas. Limite maximo
           ↳ atingido.\n");
989         pressionarEnterParaContinuar();
990         return;
991     }
992     if (numSessoes == 0 || numJogos == 0 || numJogadores == 0) {
993         printf("\nErro: E necessario ter ao menos uma Sessao, Jogo e Jogador
           ↳ cadastrado para registrar uma partida.\n");
994         pressionarEnterParaContinuar();
995         return;
996     }
997
998     printf("\n>> Registrando Partida Realizada...\n");
999     Partida novaPartida;
1000     novaPartida.idPartida = numPartidas + 1;
1001
1002     int sessaoEncontrada = 0;
1003     do {
1004         listarSessoesSimples();
1005         novaPartida.idSessaoAssociada = lerInteiro("Digite o ID da Sessao
           ↳ onde a partida ocorreu: ");
1006
1007         sessaoEncontrada = 0;
1008         for (int i = 0; i < numSessoes; i++) {
1009             if (listaSessoes[i].idSessao == novaPartida.idSessaoAssociada) {
1010                 sessaoEncontrada = 1;
1011                 break;
1012             }
1013         }
1014         if (!sessaoEncontrada) {
1015             printf("Erro: Sessao com ID %d nao encontrada. Tente novamente.\n
           ↳ n", novaPartida.idSessaoAssociada);
1016         }
1017     } while (!sessaoEncontrada);
1018
1019     int jogoValido = 0;
1020     do {
1021         listarJogosSimples();
1022         novaPartida.idJogo = lerInteiro("Digite o ID do Jogo que foi jogado:
           ↳ ");
1023
1024         if (strcmp(buscarTituloJogoPorId(novaPartida.idJogo), "Jogo base nao
           ↳ encontrado") != 0) {
1025             jogoValido = 1;
1026         } else {
1027             printf("Erro: Jogo com ID %d nao encontrado.\n", novaPartida.
           ↳ idJogo);
```

```
1028     }
1029 } while (!jogoValido);
1030
1031 do {
1032     novaPartida.duracaoMinutos = lerInteiro("Duracao da partida (em
1033         ↪ minutos): ");
1034     if (novaPartida.duracaoMinutos <= 0) {
1035         printf("Erro: A duracao deve ser maior que 0 minutos.\n");
1036     }
1037 } while (novaPartida.duracaoMinutos <= 0);
1038
1039 printf("Resultado/Pontuacao Final (texto livre): ");
1040 fgets(novaPartida.pontuacaoFinal, 200, stdin);
1041 removerNovaLinha(novaPartida.pontuacaoFinal);
1042
1043 printf("\n-- Adicionar Participantes --\n");
1044 novaPartida.numParticipantes = 0;
1045 int idParticipante;
1046
1047 do {
1048     if (novaPartida.numParticipantes >= 20) {
1049         printf("Numero maximo de participantes (20) atingido.\n");
1050         break;
1051     }
1052
1053     listarJogadoresSimples();
1054     idParticipante = lerInteiro("Digite o ID de um jogador participante
1055         ↪ (ou 0 para parar): ");
1056
1057     if (idParticipante > 0) {
1058         if (strcmp(buscarApelidoJogadorPorId(idParticipante), "Nao
1059             ↪ encontrado") == 0) {
1060             printf("Erro: Jogador com ID %d nao existe.\n",
1061                 ↪ idParticipante);
1062             continue;
1063         }
1064
1065         int jaAdicionado = 0;
1066         for (int k = 0; k < novaPartida.numParticipantes; k++) {
1067             if (novaPartida.participantes[k] == idParticipante) {
1068                 jaAdicionado = 1;
1069                 break;
1070             }
1071         }
1072
1073         if (jaAdicionado) {
1074             printf("Erro: Este jogador ja foi adicionado a partida.\n");
1075         } else {
```

```
1072         novaPartida.participantes[novaPartida.numParticipantes] =
           ↪ idParticipante;
1073         novaPartida.numParticipantes++;
1074         printf("Jogador adicionado!\n");
1075     }
1076 }
1077 } while (idParticipante != 0);
1078
1079 if (novaPartida.numParticipantes > 0) {
1080     printf("\n-- Selecionar Vencedor --\n");
1081     printf("Participantes desta partida:\n");
1082     for (int i = 0; i < novaPartida.numParticipantes; i++) {
1083         printf("ID: %d (%s)\n", novaPartida.participantes[i],
1084             buscarApelidoJogadorPorId(novaPartida.participantes[i]));
1085     }
1086
1087     int vencedorValido = 0;
1088     do {
1089         novaPartida.idVencedor = lerInteiro("Digite o ID do Vencedor (ou
           ↪ 0 para empate/jogo cooperativo): ");
1090
1091         if (novaPartida.idVencedor == 0) {
1092             vencedorValido = 1;
1093         } else {
1094             for (int i = 0; i < novaPartida.numParticipantes; i++) {
1095                 if (novaPartida.participantes[i] == novaPartida.
           ↪ idVencedor) {
1096                     vencedorValido = 1;
1097                     break;
1098                 }
1099             }
1100             if (!vencedorValido) {
1101                 printf("Erro: O ID %d nao corresponde a nenhum
           ↪ participante desta partida.\n", novaPartida.
           ↪ idVencedor);
1102             }
1103         }
1104     } while (!vencedorValido);
1105
1106 } else {
1107     novaPartida.idVencedor = 0;
1108 }
1109
1110 listaPartidas[numPartidas] = novaPartida;
1111 numPartidas++;
1112 printf("\nPartida registrada com sucesso com o ID %d!\n", novaPartida.
           ↪ idPartida);
1113 pressionarEnterParaContinuar();
```

```
1114 }
1115
1116 void avaliarJogo() {
1117     if (numAvaliacoes >= MAX_ITENS) {
1118         printf("\nNao e possivel registrar mais avaliacoes. Limite maximo
1119             ↪ atingido.\n");
1119         pressionarEnterParaContinuar();
1120         return;
1121     }
1122     if (numJogos == 0 || numJogadores == 0) {
1123         printf("\nErro: E necessario ter ao menos um Jogo e um Jogador
1124             ↪ cadastrado para criar uma avaliacao.\n");
1124         pressionarEnterParaContinuar();
1125         return;
1126     }
1127
1128     printf("\n>> Avaliando um Jogo do Catalogo...\n");
1129     Avaliacao novaAvaliacao;
1130     novaAvaliacao.idAvaliacao = numAvaliacoes + 1;
1131
1132     int jogoValido = 0;
1133     do {
1134         listarJogosSimples();
1135         novaAvaliacao.idJogoAvaliado = lerInteiro("Digite o ID do Jogo que
1136             ↪ deseja avaliar: ");
1137
1137         if (strcmp(buscarTituloJogoPorId(novaAvaliacao.idJogoAvaliado), "
1138             ↪ Jogo base nao encontrado") != 0) {
1138             jogoValido = 1;
1139         } else {
1140             printf("Erro: Jogo com ID %d nao encontrado. Tente novamente.\n"
1141                 ↪ , novaAvaliacao.idJogoAvaliado);
1141         }
1142     } while (!jogoValido);
1143
1144     int jogadorValido = 0;
1145     do {
1146         listarJogadoresSimples();
1147         novaAvaliacao.idJogadorAvaliador = lerInteiro("Digite o seu ID de
1148             ↪ Jogador (para identificar o autor da avaliacao): ");
1149
1149         if (strcmp(buscarApelidoJogadorPorId(novaAvaliacao.
1150             ↪ idJogadorAvaliador), "Nao encontrado") != 0) {
1150             jogadorValido = 1;
1151         } else {
1152             printf("Erro: Jogador com ID %d nao encontrado. Tente novamente
1153                 ↪ .\n", novaAvaliacao.idJogadorAvaliador);
1153         }
1153     }
```

```
1154 } while (!jogadorValido);
1155
1156 printf("Data da Avaliacao (DD/MM/AAAA): ");
1157 fgets(novaAvaliacao.dataAvaliacao, 15, stdin);
1158 removerNovaLinha(novaAvaliacao.dataAvaliacao);
1159
1160 do {
1161     novaAvaliacao.nota = lerFloat("Qual sua nota para este jogo (de 0.0
    ↪ a 10.0)? ");
1162     if (novaAvaliacao.nota < 0.0f || novaAvaliacao.nota > 10.0f) {
1163         printf("Erro: A nota deve ser um valor entre 0.0 e 10.0.\n");
1164     }
1165 } while (novaAvaliacao.nota < 0.0f || novaAvaliacao.nota > 10.0f);
1166
1167 do {
1168     novaAvaliacao.recomendaria = lerInteiro("Voce recomendaria este jogo
    ↪ a outros? (1=Sim, 0=Nao): ");
1169     if (novaAvaliacao.recomendaria < 0 || novaAvaliacao.recomendaria >
    ↪ 1) {
1170         printf("Erro: Digite 1 para Sim ou 0 para Nao.\n");
1171     }
1172 } while (novaAvaliacao.recomendaria < 0 || novaAvaliacao.recomendaria
    ↪ > 1);
1173
1174 do {
1175     novaAvaliacao.ehAnonima = lerInteiro("Sua avaliacao deve ser anonima
    ↪ ? (1=Sim, 0=Nao): ");
1176     if (novaAvaliacao.ehAnonima < 0 || novaAvaliacao.ehAnonima > 1) {
1177         printf("Erro: Digite 1 para Sim ou 0 para Nao.\n");
1178     }
1179 } while (novaAvaliacao.ehAnonima < 0 || novaAvaliacao.ehAnonima > 1);
1180
1181 printf("Deixe seu comentario (resenha) sobre o jogo: ");
1182 fgets(novaAvaliacao.comentario, 500, stdin);
1183 removerNovaLinha(novaAvaliacao.comentario);
1184
1185 listaAvaliacoes[numAvaliacoes] = novaAvaliacao;
1186 numAvaliacoes++;
1187 printf("\nAvaliacao para o jogo ID %d registrada com sucesso!\n",
    novaAvaliacao.idJogoAvaliado);
1188 pressionarEnterParaContinuar();
1189 }
1190
1191
1192 void gerenciarVotacoes() {
1193     int subOpcao;
1194     do {
1195         printf("\n--- Gerenciar Votacoes ---\n");
1196         printf("[1] Criar Nova Votacao para uma Sessao\n");
```

```
1197     printf("[2] Listar Votacoes Abertas\n");
1198     printf("[3] Votar em uma Enquete\n");
1199     printf("[4] Ver Resultados de uma Votacao\n");
1200     printf("[5] Voltar ao Menu Principal\n");
1201     printf("Escolha uma opcao: ");
1202     if (scanf("%d", &subOpcao) != 1) {
1203         subOpcao = -1;
1204     }
1205     limparBuffer();
1206     switch (subOpcao) {
1207     case 1:
1208         criarNovaVotacao();
1209         break;
1210     case 2:
1211         listarVotacoesAbertas();
1212         break;
1213     case 3:
1214         votarEmEnquete();
1215         break;
1216     case 4:
1217         verResultadosVotacao();
1218         break;
1219     case 5:
1220         printf("\nRetornando ao menu principal...\n");
1221         break;
1222     default:
1223         printf("\nOpcao invalida!\n");
1224         break;
1225     }
1226     if (subOpcao != 5)
1227         pressionarEnterParaContinuar();
1228 } while (subOpcao != 5);
1229 }
1230
1231 void criarNovaVotacao() {
1232     if (numVotacoes >= MAX_ITENS) {
1233         printf("\nNao e possivel criar mais votacoes. Limite maximo atingido
1234             ↪ .\n");
1235         return;
1236     }
1237     if (numSessoes == 0) {
1238         printf("\nErro: E necessario ter ao menos uma sessao agendada para
1239             ↪ criar uma votacao.\n");
1240         return;
1241     }
1242     printf("\n>> Criando Nova Votacao...\n");
1243     Votacao novaVotacao;
```

```
1243     novaVotacao.idVotacao = numVotacoes + 1;
1244
1245     int sessaoEncontrada = 0;
1246     do {
1247         listarSesoesSimples();
1248         novaVotacao.idSessaoAssociada = lerInteiro("Digite o ID da Sessao a
1249             ↳ qual esta votacao pertence: ");
1250
1251         sessaoEncontrada = 0;
1252         for (int i = 0; i < numSesoes; i++) {
1253             if (listaSesoes[i].idSessao == novaVotacao.idSessaoAssociada) {
1254                 sessaoEncontrada = 1;
1255                 break;
1256             }
1257         }
1258         if (!sessaoEncontrada) {
1259             printf("Erro: Sessao com ID %d nao encontrada. Tente novamente.\n
1260                 ↳ n", novaVotacao.idSessaoAssociada);
1261         }
1262     } while (!sessaoEncontrada);
1263
1264     printf("Qual a pergunta da enquete?: ");
1265     fgets(novaVotacao.pergunta, 200, stdin);
1266     removerNovaLinha(novaVotacao.pergunta);
1267
1268     printf("Data de Encerramento (DD/MM/AAAA): ");
1269     fgets(novaVotacao.dataEncerramento, 15, stdin);
1270     removerNovaLinha(novaVotacao.dataEncerramento);
1271
1272     do {
1273         novaVotacao.numOpcoes = lerInteiro("Quantas opcoes de voto (de 2 a
1274             ↳ 5)? ");
1275         if (novaVotacao.numOpcoes < 2 || novaVotacao.numOpcoes > 5) {
1276             printf("Erro: Numero de opcoes invalido. A votacao deve ter
1277                 ↳ entre 2 e 5 opcoes.\n");
1278         }
1279     } while (novaVotacao.numOpcoes < 2 || novaVotacao.numOpcoes > 5);
1280
1281     for (int i = 0; i < novaVotacao.numOpcoes; i++) {
1282         printf("Texto da opcao [%d]: ", i + 1);
1283         fgets(novaVotacao.opcoesTexto[i], 100, stdin);
1284         removerNovaLinha(novaVotacao.opcoesTexto[i]);
1285         novaVotacao.votosPorOpcao[i] = 0;
1286     }
1287
1288     strcpy(novaVotacao.status, "Aberta");
1289     novaVotacao.numVotantes = 0;
1290     listaVotacoes[numVotacoes] = novaVotacao;
```

```
1287     numVotacoes++;
1288     printf("\nVotacao criada com sucesso com o ID %d!\n", novaVotacao.
        ↪ idVotacao);
1289 }
1290
1291 void listarVotacoesAbertas() {
1292     printf("\n--- Votacoes Abertas ---\n");
1293     int encontradas = 0;
1294     for (int i = 0; i < numVotacoes; i++) {
1295         if (strcmp(listaVotacoes[i].status, "Aberta") == 0) {
1296             printf("-----\n");
1297             printf("ID da Votacao: %d | Sessao ID: %d\n", listaVotacoes[i].
        ↪ idVotacao,
1298                 listaVotacoes[i].idSessaoAssociada);
1299             printf("Pergunta: %s\n", listaVotacoes[i].pergunta);
1300             encontradas++;
1301         }
1302     }
1303     if (encontradas == 0) {
1304         printf("Nenhuma votacao aberta no momento.\n");
1305     }
1306     printf("-----\n");
1307 }
1308
1309 void verResultadosVotacao() {
1310     if (numVotacoes == 0) {
1311         printf("\nNenhuma votacao registrada no sistema.\n");
1312         return;
1313     }
1314
1315     int idVotacao = lerInteiro("Digite o ID da votacao para ver os
        ↪ resultados: ");
1316
1317     int indiceVotacao = -1;
1318     for (int i = 0; i < numVotacoes; i++) {
1319         if (listaVotacoes[i].idVotacao == idVotacao) {
1320             indiceVotacao = i;
1321             break;
1322         }
1323     }
1324
1325     if (indiceVotacao == -1) {
1326         printf("\nErro: Votacao com ID %d nao encontrada.\n", idVotacao);
1327         return;
1328     }
1329
1330     Votacao v = listaVotacoes[indiceVotacao];
1331     printf("\n--- Resultados para a Votacao ID %d ---\n", v.idVotacao);
```

```
1332     printf("Status: %s\n", v.status);
1333     printf("Pergunta: %s\n", v.pergunta);
1334     for (int i = 0; i < v.numOpcoes; i++) {
1335         printf("Opcao [%d]: %s - Votos: %d\n", i + 1, v.opcoesTexto[i],
1336             v.votosPorOpcao[i]);
1337     }
1338     printf("Total de Votos: %d\n", v.numVotantes);
1339     printf("-----\n");
1340 }
1341
1342 void votarEmEnquete() {
1343     if (numVotacoes == 0) {
1344         printf("\nNenhuma votacao registrada.\n");
1345         return;
1346     }
1347
1348     int idVotacao, idJogador, opcaoVoto;
1349     int indiceVotacao = -1;
1350
1351     int votacaoValida = 0;
1352     do {
1353         listarVotacoesAbertas();
1354         idVotacao = lerInteiro("Digite o ID da votacao em que deseja votar:
1355             ↪ ");
1356
1357         indiceVotacao = -1;
1358         for (int i = 0; i < numVotacoes; i++) {
1359             if (listaVotacoes[i].idVotacao == idVotacao) {
1360                 indiceVotacao = i;
1361                 break;
1362             }
1363         }
1364
1365         if (indiceVotacao == -1) {
1366             printf("Erro: Votacao com ID %d nao encontrada. Tente novamente
1367                 ↪ .\n", idVotacao);
1368         } else if (strcmp(listaVotacoes[indiceVotacao].status, "Aberta") !=
1369             ↪ 0) {
1370             printf("Erro: Esta votacao ja esta encerrada/fechada. Escolha
1371                 ↪ outra.\n");
1372             indiceVotacao = -1;
1373         } else {
1374             votacaoValida = 1;
1375         }
1376     } while (!votacaoValida);
1377
1378     printf("\n--- Votando em: %s ---\n", listaVotacoes[indiceVotacao].
1379         ↪ pergunta);
```

```
1375     for (int i = 0; i < listaVotacoes[indiceVotacao].numOpcoes; i++) {
1376         printf("Digite [%d] para: %s\n", i + 1, listaVotacoes[indiceVotacao
1377             ↪ ].opcoesTexto[i]);
1378     }
1379     int jogadorValido = 0;
1380     do {
1381         listarJogadoresSimples();
1382         idJogador = lerInteiro("Digite o seu ID de jogador para registrar o
1383             ↪ voto: ");
1384         if (strcmp(buscarApelidoJogadorPorId(idJogador), "Nao encontrado")
1385             ↪ == 0) {
1386             printf("Erro: Jogador com ID %d nao encontrado no sistema.\n",
1387                 ↪ idJogador);
1388             continue;
1389         }
1390         int jaVotou = 0;
1391         for (int i = 0; i < listaVotacoes[indiceVotacao].numVotantes; i++) {
1392             if (listaVotacoes[indiceVotacao].votantes[i] == idJogador) {
1393                 jaVotou = 1;
1394                 break;
1395             }
1396         }
1397         if (jaVotou) {
1398             printf("Erro: O jogador ID %d ja votou nesta enquete. Tente
1399                 ↪ outro ID.\n", idJogador);
1400         } else {
1401             jogadorValido = 1;
1402         }
1403     } while (!jogadorValido);
1404     char promptVoto[50];
1405     sprintf(promptVoto, "Escolha sua opcao (1 a %d): ", listaVotacoes[
1406         ↪ indiceVotacao].numOpcoes);
1407     do {
1408         opcaoVoto = lerInteiro(promptVoto);
1409         if (opcaoVoto < 1 || opcaoVoto > listaVotacoes[indiceVotacao].
1410             ↪ numOpcoes) {
1411             printf("Erro: Opcao invalida. Digite um numero entre 1 e %d.\n",
1412                 ↪ listaVotacoes[indiceVotacao].numOpcoes);
1413         }
1414     } while (opcaoVoto < 1 || opcaoVoto > listaVotacoes[indiceVotacao].
1415         ↪ numOpcoes);
```

```
1414     listaVotacoes[indiceVotacao].votosPorOpcao[opcaoVoto - 1]++;
1415     listaVotacoes[indiceVotacao].votantes[listaVotacoes[indiceVotacao].
    ↪ numVotantes] = idJogador;
1416     listaVotacoes[indiceVotacao].numVotantes++;
1417     printf("\nVoto computado com sucesso!\n");
1418 }
1419
1420 void gerenciarCardapio() {
1421     int subOpcao;
1422     do {
1423         printf("\n--- Gerenciar Cardapio das Sessoes ---\n");
1424         printf("[1] Adicionar Item ao Cardapio de uma Sessao\n");
1425         printf("[2] Ver Cardapio Completo de uma Sessao\n");
1426         printf("[3] Voltar ao Menu Principal\n");
1427         printf("Escolha uma opcao: ");
1428         if (scanf("%d", &subOpcao) != 1) {
1429             subOpcao = -1;
1430         }
1431         limparBuffer();
1432         switch (subOpcao) {
1433             case 1:
1434                 adicionarItemAoCardapio();
1435                 break;
1436             case 2:
1437                 verCardapioDaSessao();
1438                 break;
1439             case 3:
1440                 printf("\nRetornando ao menu principal...\n");
1441                 break;
1442             default:
1443                 printf("\nOpcao invalida!\n");
1444                 break;
1445         }
1446         if (subOpcao != 3)
1447             pressionarEnterParaContinuar();
1448     } while (subOpcao != 3);
1449 }
1450
1451 void adicionarItemAoCardapio() {
1452     if (numCardapioItens >= MAX_ITENS * 5) {
1453         printf("\nNao e possivel adicionar mais itens ao cardapio. Limite
    ↪ maximo atingido.\n");
1454         return;
1455     }
1456     if (numSessoes == 0 || numJogadores == 0) {
1457         printf("\nErro: E necessario ter ao menos uma Sessao e um Jogador
    ↪ cadastrado.\n");
1458         return;
```

```
1459     }
1460
1461     printf("\n>> Adicionando Item ao Cardapio...\n");
1462     CardapioItem novoItem;
1463     novoItem.idCardapioItem = numCardapioItens + 1;
1464
1465     int sessaoEncontrada = 0;
1466     do {
1467         listarSesoesSimples();
1468         novoItem.idSessaoAssociada = lerInteiro("Digite o ID da Sessao para
1469             ↪ adicionar o item: ");
1470
1471         sessaoEncontrada = 0;
1472         for (int i = 0; i < numSesoes; i++) {
1473             if (listaSesoes[i].idSessao == novoItem.idSessaoAssociada) {
1474                 sessaoEncontrada = 1;
1475                 break;
1476             }
1477         }
1478         if (!sessaoEncontrada) {
1479             printf("Erro: Sessao com ID %d nao encontrada. Tente novamente.\n
1480                 ↪ ", novoItem.idSessaoAssociada);
1481         }
1482     } while (!sessaoEncontrada);
1483
1484     int jogadorValido = 0;
1485     do {
1486         listarJogadoresSimples();
1487         novoItem.idJogadorResponsavel = lerInteiro("Digite o ID do Jogador
1488             ↪ responsavel por este item: ");
1489
1490         if (strcmp(buscarApelidoJogadorPorId(novoItem.idJogadorResponsavel),
1491             ↪ "Nao encontrado") != 0) {
1492             jogadorValido = 1;
1493         } else {
1494             printf("Erro: Jogador com ID %d nao encontrado. Tente novamente
1495                 ↪ .\n", novoItem.idJogadorResponsavel);
1496         }
1497     } while (!jogadorValido);
1498
1499     printf("Nome do Item: ");
1500     fgets(novoItem.nomeItem, 100, stdin);
1501     removerNovaLinha(novoItem.nomeItem);
1502
1503     printf("Quantidade: ");
1504     fgets(novoItem.quantidade, 50, stdin);
1505     removerNovaLinha(novoItem.quantidade);
```

```
1502     printf("Categoria (Salgado, Doce, Bebida, etc.): ");
1503     fgets(novoItem.categoria, 50, stdin);
1504     removerNovaLinha(novoItem.categoria);
1505
1506     printf("Notas Adicionais: ");
1507     fgets(novoItem.notasAdicionais, 200, stdin);
1508     removerNovaLinha(novoItem.notasAdicionais);
1509
1510     strcpy(novoItem.status, "Confirmado");
1511     listaCardapio[numCardapioItens] = novoItem;
1512     numCardapioItens++;
1513     printf("\nItem adicionado ao cardapio da sessao com sucesso!\n");
1514 }
1515
1516 void verCardapioDaSessao() {
1517     if (numSesoes == 0) {
1518         printf("\nNenhuma sessao agendada para ver o cardapio.\n");
1519         return;
1520     }
1521     int idSessao;
1522     int sessaoEncontrada = 0;
1523
1524     do {
1525         listarSesoesSimples();
1526         idSessao = lerInteiro("Digite o ID da Sessao para ver o cardapio: ")
1527             ↪ ;
1528
1529         sessaoEncontrada = 0;
1530         for (int i = 0; i < numSesoes; i++) {
1531             if (listaSesoes[i].idSessao == idSessao) {
1532                 sessaoEncontrada = 1;
1533                 break;
1534             }
1535         }
1536
1537         if (!sessaoEncontrada) {
1538             printf("Erro: Sessao com ID %d nao encontrada. Tente novamente.\n
1539                 ↪ n", idSessao);
1540         }
1541     } while (!sessaoEncontrada);
1542
1543     printf("\n--- Cardapio da Sessao ID %d ---\n", idSessao);
1544     int encontrados = 0;
1545     for (int i = 0; i < numCardapioItens; i++) {
1546         if (listaCardapio[i].idSessaoAssociada == idSessao) {
1547             CardapioItem item = listaCardapio[i];
1548             printf("-----\n");
1549             printf("Item:           %s (%s)\n", item.nomeItem, item.quantidade);
```

```
1548     printf("Categoria:      %s\n", item.categoria);
1549     printf("Responsavel:   ID %d (%s)\n", item.idJogadorResponsavel,
1550           buscarApelidoJogadorPorId(item.idJogadorResponsavel));
1551     printf("Notas:         %s\n", item.notasAdicionais);
1552     encontrados++;
1553 }
1554 }
1555 if (encontrados == 0) {
1556     printf("Nenhum item de cardapio registrado para esta sessao.\n");
1557 }
1558 printf("-----\n");
1559 }
1560
1561 int lerInteiro(const char *prompt) {
1562     int valor;
1563     int resultadoScan;
1564
1565     do {
1566         printf("%s", prompt);
1567         resultadoScan = scanf("%d", &valor);
1568
1569         if (resultadoScan != 1) {
1570             printf("\nEntrada invalida! Por favor, digite um numero inteiro.\n
1571                 ↪ \n");
1572         }
1573
1574         limparBuffer();
1575     } while (resultadoScan != 1);
1576
1577     return valor;
1578 }
1579
1580 float lerFloat(const char *prompt) {
1581     float valor;
1582     int resultadoScan;
1583
1584     do {
1585         printf("%s", prompt);
1586         resultadoScan = scanf("%f", &valor);
1587
1588         if (resultadoScan != 1) {
1589             printf("\nEntrada invalida! Por favor, digite um numero.\n\n");
1590         }
1591
1592         limparBuffer();
1593     } while (resultadoScan != 1);
```

```
1595
1596     return valor;
1597 }
1598
1599 void limparBuffer() {
1600     int c;
1601     while ((c = getchar()) != '\n' && c != EOF);
1602 }
1603
1604 void pressionarEnterParaContinuar() {
1605     printf("\nPressione Enter para continuar...");
1606     getchar();
1607 }
1608
1609 void removerNovaLinha(char *str) {
1610     size_t len = strlen(str);
1611     if (len > 0 && str[len - 1] == '\n') {
1612         str[len - 1] = '\0';
1613     }
1614 }
1615
1616 const char *buscarApelidoJogadorPorId(int jogadorId) {
1617     for (int i = 0; i < numJogadores; i++) {
1618         if (listaJogadores[i].idJogador == jogadorId) {
1619             return listaJogadores[i].apelido;
1620         }
1621     }
1622     return "Nao encontrado";
1623 }
1624
1625 const char *buscarTituloJogoPorId(int jogoId) {
1626     for (int i = 0; i < numJogos; i++) {
1627         if (listaJogos[i].idJogo == jogoId) {
1628             return listaJogos[i].titulo;
1629         }
1630     }
1631     return "Jogo base nao encontrado";
1632 }
1633
1634 const char *buscarNomeLocalPorId(int localId) {
1635     for (int i = 0; i < numLocais; i++) {
1636         if (listaLocais[i].idLocal == localId) {
1637             return listaLocais[i].nomeLocal;
1638         }
1639     }
1640     return "Local nao encontrado";
1641 }
1642
```

```
1643 void listarJogadoresSimples() {
1644     printf("\n-- Jogadores Cadastrados --\n");
1645     if (numJogadores == 0) {
1646         printf("Nenhum jogador cadastrado.\n");
1647     } else {
1648         for (int i = 0; i < numJogadores; i++) {
1649             printf("ID: %d | Nome: %s | Apelido: %s\n", listaJogadores[i].
1650                 ↪ idJogador,
1651                 listaJogadores[i].nomeCompleto, listaJogadores[i].apelido);
1652         }
1653     }
1654     printf("-----\n");
1655 }
1656 void listarJogosSimples() {
1657     printf("\n-- Jogos no Catalogo --\n");
1658     if (numJogos == 0) {
1659         printf("Nenhum jogo cadastrado.\n");
1660     } else {
1661         for (int i = 0; i < numJogos; i++) {
1662             printf("ID: %d | Titulo: %s\n", listaJogos[i].idJogo,
1663                 listaJogos[i].titulo);
1664         }
1665     }
1666     printf("-----\n");
1667 }
1668 void listarExpansoesSimples() {
1669     printf("\n-- Expansoes no Catalogo --\n");
1670     if (numExpansoes == 0) {
1671         printf("Nenhuma expansao cadastrada.\n");
1672     } else {
1673         for (int i = 0; i < numExpansoes; i++) {
1674             printf("ID: %d | Titulo: %s (Para Jogo ID: %d)\n",
1675                 listaExpansoes[i].idExpansao, listaExpansoes[i].
1676                 ↪ tituloExpansao,
1677                 listaExpansoes[i].idJogoBase);
1678         }
1679     }
1680     printf("-----\n");
1681 }
1682 void listarSesoesSimples() {
1683     printf("\n-- Sesoes Agendadas --\n");
1684     if (numSesoes == 0) {
1685         printf("Nenhuma sessao agendada.\n");
1686     } else {
```

```

1689     for (int i = 0; i < numSessoes; i++) {
1690         printf("ID: %d | Evento: %s | Data: %s\n", listaSessoes[i].
            ↪ idSessao,
1691             listaSessoes[i].nomeEvento, listaSessoes[i].data);
1692     }
1693 }
1694 printf("-----\n");
1695 }
1696
1697 void listarCampanhasSimples() {
1698     printf("\n-- Campanhas em Andamento --\n");
1699     if (numCampanhas == 0) {
1700         printf("Nenhuma campanha criada.\n");
1701     } else {
1702         for (int i = 0; i < numCampanhas; i++) {
1703             printf("ID: %d | Nome: %s\n", listaCampanhas[i].idCampanha,
1704                 listaCampanhas[i].nomeCampanha);
1705         }
1706     }
1707     printf("-----\n");
1708 }
1709
1710 void listarLocaisSimples() {
1711     printf("\n-- Locais Cadastrados --\n");
1712     if (numLocais == 0) {
1713         printf("Nenhum local cadastrado.\n");
1714     } else {
1715         for (int i = 0; i < numLocais; i++) {
1716             printf("ID: %d | Nome: %s\n", listaLocais[i].idLocal,
1717                 listaLocais[i].nomeLocal);
1718         }
1719     }
1720     printf("-----\n");
1721 }

```